

IMPERIAL

Imperial College London

MSc in Applied Mathematics

MSc Project

**Optimal Control of the Poisson and Burgers
Equations, with a Study of Mesh-Dependence
in the Optimisation**

Author:

Bert Jim Chang

`bert-jim.chang25@imperial.ac.uk`

Supervisor:

Professor David A. Ham

Department of Mathematics

Faculty of Natural Sciences

August 11, 2026

Abstract

This dissertation studies mesh dependence in PDE-constrained optimisation. The number of iterations a gradient-based optimiser needs can stay flat as the computational mesh is refined, or grow without bound. Which of the two happens depends on the inner product the optimiser uses to turn a derivative into a gradient. We first derive and verify the adjoint machinery for two model problems, the linear Poisson equation and the nonlinear, time-dependent viscous Burgers equation, using manufactured solutions and Taylor tests. We then reproduce the finding of Schwedes et al. [13] that respecting the function-space L^2 inner product through its Riesz map makes a quasi-Newton optimiser essentially mesh-independent, while the Euclidean inner product used by default in most optimisation packages does not. A survey of the optimisers in PETSc TAO establishes which of them honour that inner product, and a comparison of four L-BFGS implementations at source level explains why one of them appeared not to. The main contribution of this work is to show that the same mesh dependence appears again one level deeper, in the inner Krylov solve that a Newton optimiser uses to compute each step. We remove it there with the same Riesz map, applied as a preconditioner. The inner iteration count falls from over a thousand to a constant twelve across the mesh sweep. The metric therefore has to be supplied at both the outer and the inner level. The idea that an inner product can act as a preconditioner is already present in classical operator-preconditioning theory, and has been raised in discussion among PETSc/TAO developers. We appear to be the first to expose and remove the effect experimentally in an adjoint optimisation pipeline, and the first to implement the fix in pyadjoint, the adjoint framework of Firedrake. We explain spectrally why the Riesz map is the right preconditioner, show that it must match the norm in which the control is regularised, and identify one solver that neither mechanism can reach. A production version of the preconditioner has been merged upstream.

Acknowledgements

I would like to thank my supervisor, Professor David A. Ham, for his patience and the time he gave to this project. His guidance in our discussions greatly helped with the direction that this dissertation took, and I have learnt a great deal from his insights.

I would also like to thank Josh Hope-Collins for joining some of these meetings, and for his help with the pyadjoint implementation. Thanks also go to the developers on the PETSc Discord server, whose discussions of inner products and preconditioning pointed towards the central result of this work.

Declaration of Originality

The work contained in this thesis is my own work unless otherwise stated.

Contents

1	Introduction	1
1.1	Contributions	2
2	Mathematical Preliminaries	3
2.1	Partial differential equations	3
2.2	The PDE-constrained optimisation problem	3
2.3	Domain and function spaces	3
2.4	Dual spaces and the Riesz representation theorem	5
2.5	Differential calculus in function spaces	6
2.6	Bilinear forms and weak formulations	7
3	The Forward Poisson Problem	7
3.1	Strong formulation	7
3.2	Derivation of the weak form	8
4	The Poisson Optimal Control Problem	9
4.1	Objective functional	9
4.2	The constrained optimisation problem	9
4.3	Reduced formulation	10
5	Derivative of the Reduced Functional and the Adjoint Equation	10
5.1	The general derivation	11
5.2	The Poisson tangent linear equation	12
5.3	Partial derivatives of the Poisson objective functional	13
5.4	The Poisson adjoint problem	13
5.5	Derivative of the Poisson reduced functional	14
5.6	The L^2 -gradient	14
6	Numerical Implementation and Verification of the Poisson Problem	14
6.1	Forward Poisson by hand	15
6.2	Forward Poisson solve in Firedrake	16
6.3	Manufactured solution and convergence study	17
6.4	Adjoint solve and gradient computation	19
6.5	Taylor test for gradient verification	20
7	The Forward Problem for the Scalar Viscous Burgers Equation	20
7.1	Strong formulation	20
7.2	Weak formulation	21
8	Derivative of the Reduced Functional for the Burgers Problem	21
8.1	Time discretisation and the composed forward problem	22
8.2	The tangent linear system	23
8.3	The adjoint system	23
8.4	Derivative of the reduced functional and the gradient	25
9	Numerical Implementation and Verification of the Burgers Problem	26
9.1	Scalar Burgers numerically	26
9.2	Manufactured solution and convergence study	26
9.3	Discrete adjoint solve and gradient computation	28
9.4	Taylor test for gradient verification	28

10 Mesh Dependence in PDE-Constrained Optimisation	29
10.1 The two Riesz maps	29
10.2 Non-uniform meshes and the mesh ratio	30
10.3 The benchmark: Table 2.2 of Schwedes et al. [13]	31
10.4 Mesh generation	32
10.5 Implementation	34
10.6 BFGS in a Hilbert space	35
10.7 Numerical experiments: Panel (a), $H = L^2(\Omega)$	36
10.8 Numerical experiments: Panel (b), $H = H^1(\Omega)$	38
10.9 The limited-memory history in TAO/LMVM	39
10.10A survey of the TAO solvers	41
10.11Isolation of the seed in bqnl	43
10.12The reduced Hessian in the two inner products	44
11 Source-Level Comparison of the L-BFGS Implementations	45
11.1 The common skeleton	45
11.2 The two Schwedes formulations	46
11.3 The TAO/LMVM recursion	46
11.4 The Moola recursion	47
11.5 The four implementations in pseudo-code	47
11.6 Correctness of the TAO/LMVM recursion	48
12 Two-Level Riesz-Map Preconditioning	49
12.1 A mesh-dependence hidden inside TAO/NLS	49
12.2 Preconditioning of Krylov methods	50
12.3 Choosing the preconditioner	50
12.4 Preconditioning the inner CG of TAO/NLS	51
12.5 A production-ready preconditioner for pyadjoint	52
12.6 Matching the Riesz map to the regularisation	53
12.7 Use in a pyadjoint problem	54
12.8 Nonlinear conjugate gradient and the metric	55
13 Conclusions and Future Work	56
A Code availability	59

1 Introduction

Many problems in science and engineering involve determining unknown inputs to a physical system based on partial or indirect observations of its output. These sorts of problems are commonly formulated as PDE-constrained optimisation problems, in which the behaviour of the system is governed by a partial differential equation, while the unknown input is found by minimising a suitable objective functional.

The governing equations in the problems we consider are partial differential equations, and one widely used way to solve them on a computer is the finite element method. The idea is to replace the continuous domain by a *mesh*, a partition of it into many small cells, here triangles, and to approximate the unknown function by a simple polynomial on each cell. This turns the differential equation into a finite system of equations, with roughly one unknown per mesh vertex, which a computer can solve, and the approximation improves as the mesh is made finer. Importantly for what follows, practical meshes are deliberately *non-uniform*, in that they are refined where the solution changes rapidly and left coarse elsewhere. That a mesh can be graded in this way, with small cells in some regions and large cells in others, is exactly what makes mesh-dependence a phenomenon worth studying.

Solving an optimisation problem on top of such a finite element model needs several ingredients, which we build in order. The first is a *forward solve*, which, given a trial control, solves the PDE and returns the resulting state. The second is the *gradient* of the objective with respect to the control, which the optimiser needs in order to know which way to step; computing it directly, one direction at a time, would be far too expensive, so we use the *adjoint* method, which recovers the entire gradient from a single additional solve. The third is the *optimiser* itself, which uses the gradient to iterate towards the best control. Each of these has to be built and, just as importantly, verified before it can be trusted, which is why the first half of this dissertation derives and tests the forward solve, the adjoint and the gradient on two model problems before any mesh-dependence experiment is attempted. The final ingredient, and the one the whole story turns on, is the *inner product* the optimiser uses to turn the gradient into a search direction.

Such problems are solved iteratively, and one would expect that refining the computational mesh simply sharpens the answer without changing the character of the optimisation itself. This expectation can fail dramatically. On the same problem, solved by the same quasi-Newton method to the same tolerance, the number of iterations needed can either stay essentially constant as the mesh is refined or grow without bound. In the experiments of this dissertation, the two outcomes were a flat count of around six and a count that doubles at every refinement, reaching well over a thousand. The difference is not the problem, the algorithm, or the mesh, but a single choice buried inside the optimiser, namely the inner product it uses to turn a derivative into a gradient. Most optimisation packages default to the Euclidean inner product of the coefficient vector, which is the wrong choice on a non-uniform mesh. The right choice is the inner product of the function space in which the control actually lives, applied through its Riesz map. This dissertation is really the story of that one choice.

We approach it in three stages. First, we build and verify the adjoint-based optimisation machinery on two model problems, the linear Poisson equation and the nonlinear, time-dependent Burgers equation, so that the later experiments rest on a pipeline whose forward solves and gradients are known to be correct. Second, we reproduce the result of Schwedes et al. [13] that respecting the function-space inner product makes a quasi-Newton optimiser essentially mesh-independent, while the Euclidean inner product does not. Third, we expose the same mesh-dependence one level deeper, in the inner Krylov solve that a Newton optimiser uses to compute each step, and remove it by the same idea applied there, as a preconditioner. The idea that the inner product can act as a preconditioner is already

present in classical operator preconditioning [6, 8] and has been discussed among PETSc/TAO developers [7]; the contribution here is to expose and remove the effect experimentally in an adjoint optimisation pipeline, to explain it spectrally, and to contribute a production preconditioner that has since been merged into pyadjoint, together with a regression test merged into Firedrake. The metric has to be supplied at both levels.

The dissertation is organised as follows. Section 2 collects the functional-analytic background (Sobolev spaces, dual spaces, and the Riesz representation theorem) on which the whole argument rests. Sections 3 to 6 take the linear Poisson problem through its weak form, its optimal-control formulation, the adjoint derivation of the gradient, and numerical verification, and Sections 7 to 9 repeat the same programme for the nonlinear Burgers problem. Section 10 reproduces the mesh-dependence experiment, Section 11 traces at the level of source code exactly where the inner product enters each limited-memory optimiser, and Section 12 develops the two-level preconditioning contribution. Section 13 concludes.

1.1 Contributions

The novel contributions of this dissertation are the following.

1. A systematic determination, across a mesh-refinement sweep spanning ratios from four to 128, of which of the unconstrained and bound-constrained optimisation algorithms in the current PETSc TAO library honour the function-space inner product and which do not.
2. Verification that TAO’s limited-memory quasi-Newton method honours the inner product through its initial-Hessian seed and is mesh-independent at either history length, with the history setting only the size of the flat count; this resolves an apparent failure to reproduce the reference results of Schwedes et al. [13] that had earlier been attributed to an internal Euclidean inner product.
3. A demonstration that the Newton line-search method is mesh-independent in its outer iterations while the mesh-dependence reappears in its inner Krylov solve, and that preconditioning that inner solve with the Riesz map removes it, reducing the inner iteration count from over a thousand to a constant across the mesh sweep.
4. The principal software contribution of this work is a production-quality implementation of that inner preconditioner, **RieszMapPC**. Implemented as a PETSc Python preconditioner, it applies the Riesz map of each pyadjoint **Control** to the reduced-Hessian Krylov solve inside TAO’s Newton method, reading the metric directly from the **Control** so that a user gets a mesh-independent inner solve from the Riesz map already attached to their control, selected through two PETSc options, with none of the manual plumbing of the research prototype. Verified against the hand-wired prototype to the iteration, it has been merged into pyadjoint, so the two-level fix is available to every user of the stack rather than it being confined to this dissertation. The Firedrake regression test that exercises it end to end has been merged in turn. To our knowledge it is the first implementation to expose and remove this inner-level mesh-dependence in an adjoint optimisation pipeline.
5. A diagnosis of the two remaining mesh-dependent algorithms, namely the bounded quasi-Newton line-search method, which is curable by supplying the same seed by hand (an effect we isolate in a controlled experiment and explain through the spectrum of the reduced Hessian in the two inner products), and nonlinear conjugate gradient, which is shown to be structurally incurable within TAO for want of any interface at which the metric could be supplied.

2 Mathematical Preliminaries

Before we can derive the adjoint equation, discretise the optimisation problem, or talk about what “mesh-independent” even means, we need to establish some background, from the equations themselves to the functional-analytic objects that the mesh-dependence story rests on. Here we loosely follow the first chapter of Schwedes et al. [13].

2.1 Partial differential equations

First, we need the definition of a PDE. A PDE (partial differential equation) is an equation that relates an unknown function of several variables to its partial derivatives. For example we can have:

- The heat equation $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$, which describes how a temperature $u(x, t)$ spreads along a rod over time.
- The wave equation $\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$, which describes the displacement $u(x, t)$ of a vibrating string.

The two main model problems in this dissertation are Poisson’s equation $-\Delta u = m$, where Δ is the Laplacian (the sum of the second partial derivatives), and the viscous Burgers equation. Poisson’s equation has no time dependence and so describes a steady state, while the Burgers equation is nonlinear and time-dependent.

2.2 The PDE-constrained optimisation problem

We are interested in problems where we want to recover an unknown input to a physical system from indirect observations of its output. So, it is useful for us to fix the terminology that we will use to describe this class of problems. Every problem we study is built from the following four components:

The first is the *state variable* u . This is the quantity that describes the response of the physical system.

The second is the *control variable* m . This is the input that we are free to vary, such as a source term, a boundary value, or a coefficient. The choice of control is what distinguishes one instance of the system from another, and this is the variable that we want to optimise over.

The third is the *state equation*, the PDE that governs the system and links the state to the control. For each admissible control m , the state equation determines a corresponding state u . In this sense, the state is a function of the control (we make this dependence precise later using the solution operator of Definition 4.3).

The final component is the *objective functional* (or cost functional) $J(u, m)$, which measures how good a given state and control pair is. It usually penalises the mismatch between the state u and a prescribed desired state d , together with the size of the control. A *PDE-constrained optimisation problem* is then the problem of choosing the control m that minimises $J(u, m)$, subject to the constraint that u and m satisfy the state equation.

2.3 Domain and function spaces

Since a PDE is an equation about a function on a domain, we must first define both functions and domains properly. Poisson’s equation and the other elliptic problems of this dissertation¹ can be posed in the classical setting, where solutions are smooth enough to differentiate pointwise

¹Informally, the elliptic PDEs are the steady problems, with Poisson’s equation the prototype; the precise definition involves the second-order terms and is not needed here.

everywhere. We choose instead to work in a weaker setting, because we want to admit less regular functions, whose derivatives are defined only in a weak sense, by integration against test functions². So, we need a function space which is large enough to contain them. The Sobolev spaces $H^1(\Omega)$ and $H_0^1(\Omega)$ give us exactly this [5, Sect. 5.2].

Let $\Omega \subset \mathbb{R}^d$, with $d \in \{1, 2, 3\}$, be a bounded open domain with Lipschitz boundary $\partial\Omega$.³ We assume that all functions are defined on Ω , unless stated otherwise. We denote by $L^2(\Omega)$ the space of square-integrable functions on Ω , equipped with the inner product and norm

$$(u, v)_{L^2(\Omega)} := \int_{\Omega} uv \, dx, \quad \|u\|_{L^2(\Omega)} := (u, u)_{L^2(\Omega)}^{1/2}. \quad (1)$$

Definition 2.1 (Sobolev space $H^1(\Omega)$). The *Sobolev space* $H^1(\Omega)$ is defined by

$$H^1(\Omega) := \left\{ u \in L^2(\Omega) \mid \nabla u \in (L^2(\Omega))^d \right\}. \quad (2)$$

So, we have that a function u belongs to $H^1(\Omega)$ if both the function itself and its first (weak) derivatives are square-integrable. This space is equipped with the norm

$$\|u\|_{H^1(\Omega)}^2 := \|u\|_{L^2(\Omega)}^2 + \|\nabla u\|_{L^2(\Omega)}^2. \quad (3)$$

This norm comes from the H^1 inner product,

$$(u, v)_{H^1(\Omega)} := (u, v)_{L^2(\Omega)} + (\nabla u, \nabla v)_{L^2(\Omega)} = \int_{\Omega} uv \, dx + \int_{\Omega} \nabla u \cdot \nabla v \, dx. \quad (4)$$

To also incorporate homogeneous Dirichlet boundary conditions into the function space itself, we also want to introduce the subspace $H_0^1(\Omega)$.

Definition 2.2 (Sobolev space $H_0^1(\Omega)$). The *Sobolev space* $H_0^1(\Omega)$ is defined as the closure of $C_c^\infty(\Omega)$ in the $H^1(\Omega)$ norm.

Here, $C_c^\infty(\Omega)$ denotes the smooth functions with compact support in Ω (i.e., smooth functions which vanish in a neighbourhood of $\partial\Omega$). Taking the *closure* of this set in the H^1 norm means we also include every function that can be approximated arbitrarily well, in the H^1 norm, by such smooth functions. Functions in $H_0^1(\Omega)$ are zero on the boundary $\partial\Omega$, which makes this the natural solution space for problems with homogeneous Dirichlet boundary conditions.

They also satisfy the following inequality, which we can use to show well-posedness.

Theorem 2.3 (Poincaré inequality). *There is a constant $C_\Omega > 0$, depending only on Ω , such that $\|u\|_{L^2(\Omega)} \leq C_\Omega \|\nabla u\|_{L^2(\Omega)}$ for all $u \in H_0^1(\Omega)$ [2].*

The spaces $L^2(\Omega)$, $H^1(\Omega)$ and $H_0^1(\Omega)$ defined above are all examples of *Hilbert spaces*, and the mesh-dependence story depends on the choice of inner product that is placed on such a space. We record the two underlying notions precisely.

²A test function here is just a smooth function that we are free to choose, and the weak derivative is defined by asking the usual integration-by-parts formula to hold against every such test function, rather than by differentiating at each point. We show exactly how this works for Poisson's equation in Section 3.2; for now it is enough to know that it lets us make sense of derivatives for functions that are not smooth.

³Informally, a Lipschitz boundary is one that is not too rough, in the sense that corners like a polygon's are allowed, but not cusps or fractal-like features. The unit square used in the experiments qualifies.

Definition 2.4 (Inner product). An *inner product* on a real vector space H is a map $(\cdot, \cdot) : H \times H \rightarrow \mathbb{R}$. More precisely, let $u, v, w \in H$ be vectors and $\alpha \in \mathbb{R}$ a scalar; then it satisfies the following four properties:

1. $(u + v, w) = (u, w) + (v, w)$ (additivity)
2. $(\alpha v, w) = \alpha (v, w)$ (homogeneity)
3. $(v, w) = (w, v)$ (symmetry)
4. $(v, v) \geq 0$, with equality if and only if $v = 0$ (positive definiteness)

An inner product induces the norm $\|v\| := (v, v)^{1/2}$.

Definition 2.5 (Hilbert space). A *Hilbert space* is an inner product space that is *complete*, meaning every Cauchy sequence has a limit in the space.

Throughout the dissertation, the control variable m lives in a Hilbert space M , the control space, equipped with an inner product $(\cdot, \cdot)_M$ and associated norm $\|\cdot\|_M$.

2.4 Dual spaces and the Riesz representation theorem

As we see later in this dissertation, the mesh-dependence story here is, at its root, a story about a choice of inner product. For an inner product to be the subject of a choice, we first need to understand why we have any freedom to choose this at all. This requires understanding the relationship between a Hilbert space and its dual. The derivative of a real-valued functional on a Hilbert space is naturally an element of the dual space (a linear functional), but actually in practice we want to picture it as an element of the original space (“a gradient”). So, we introduce the Riesz representation theorem which provides a canonical bijection between the two views. This bijection depends on the choice of inner product, and that dependence is exactly what the mesh-dependence study in Section 10 depends on. Here we set up the statements that we need, and the consequences are described in Section 10.

Let H be a Hilbert space with inner product $(\cdot, \cdot)_H$.

Definition 2.6 (Dual space). The *dual space* of H , denoted H^* , is the space of all continuous linear functionals on H [13, p. 21]:

$$H^* := \{f : H \rightarrow \mathbb{R} \mid f \text{ is linear and continuous}\}. \quad (5)$$

We write duality pairings between H^* and H in angle brackets, $\langle f, v \rangle$, and inner products in H in parentheses, $(u, v)_H$, so that the bracket style can tell the reader which kind of object lives on each side. One specific case that we will use throughout the dissertation is the Sobolev dual $H^{-1}(\Omega) := H_0^1(\Omega)^*$.

Although H and H^* are different spaces of different objects, the Riesz representation theorem below gives a canonical bijection between them [13, Theorem 1.1].

Theorem 2.7 (Riesz representation theorem). *Let H be a Hilbert space with inner product $(\cdot, \cdot)_H$. For any continuous linear functional $f \in H^*$, there exists a unique $u_f \in H$ such that*

$$f(v) = (u_f, v)_H \quad \forall v \in H, \quad (6)$$

and the norms agree, i.e. $\|f\|_{H^} = \|u_f\|_H$.*

This bijection is what lets us pass between the dual view and the primal view of a linear functional, and the map that effects the bijection is called the Riesz map:

Definition 2.8 (Riesz map and Riesz representer). The map $\mathcal{R}_H : H^* \rightarrow H$ that sends each $f \in H^*$ to its unique representative $u_f \in H$ is called the *Riesz map*. The element $\mathcal{R}_H(f) := u_f$ is called the *Riesz representer* of f in H .

The Riesz map depends on the choice of inner product on H - indeed, the same linear functional has different Riesz representers under different inner products.

2.5 Differential calculus in function spaces

To minimise a functional, we have to differentiate it, and the adjoint method in particular depends on the chain rule applied to a composite operator. In finite dimensions, partial derivatives stacked into a vector are enough. However, the optimisation variable here lives in an infinite-dimensional Hilbert space, so we even have to be careful about what “direction” actually means. So, we define the Fréchet derivative as the right replacement for the finite-dimensional gradient, namely a bounded linear operator⁴ that captures the first-order change of a map between Hilbert spaces [13, Sect. 1.2.2–1.2.3].

Let U and V be Hilbert spaces, and let $F : U \rightarrow V$ be a (possibly nonlinear) operator.

Definition 2.9 (Fréchet derivative). The operator F is *Fréchet differentiable* at $u \in U$ if there exists a bounded linear operator $dF(u) \in \mathcal{L}(U, V)$, where $\mathcal{L}(U, V)$ denotes the space of bounded linear operators from U to V , such that

$$\lim_{\|\delta u\|_U \rightarrow 0} \frac{\|F(u + \delta u) - F(u) - dF(u) \delta u\|_V}{\|\delta u\|_U} = 0. \quad (7)$$

The directional derivative in the direction $\delta u \in U$ is written $dF(u; \delta u) := dF(u) \delta u$.

When an operator or functional depends on multiple variables, we can also naturally extend the idea of Fréchet differentiation to partial derivatives.

Definition 2.10 (Partial Fréchet derivative). Let $F : U \times M \rightarrow V$. The *partial Fréchet derivative* of F with respect to u at the point (u, m) , acting in the direction $\delta u \in U$, is denoted as $\partial_u F(u, m; \delta u) \in V$. It is defined as the Fréchet derivative of the map $u \mapsto F(u, m)$ with fixed m . The partial derivative with respect to m , denoted $\partial_m F(u, m; \delta m)$, is defined analogously.

Finally, we can get the derivative of a composite functional from the chain rule.

Theorem 2.11 (Chain rule). Let M and U be Hilbert spaces. Let $S : M \rightarrow U$ be Fréchet differentiable at $m \in M$, and let $J : U \times M \rightarrow \mathbb{R}$ be continuously Fréchet differentiable at $(S(m), m)$. Then the reduced functional $\tilde{J}(m) := J(S(m), m)$ is Fréchet differentiable at m , and its derivative in the direction $\delta m \in M$ is given by

$$d\tilde{J}(m; \delta m) = \partial_u J(S(m), m; dS(m; \delta m)) + \partial_m J(S(m), m; \delta m). \quad (8)$$

⁴A linear map between the two spaces whose output norm is at most a fixed multiple of the input norm.

2.6 Bilinear forms and weak formulations

Even with the function spaces and calculus in place, the PDEs we want to solve are still written in their classical, pointwise form, but this form requires too much smoothness from the solution. The standard fix is to multiply by a test function and integrate by parts, moving derivatives off the solution and onto the test function. The reformulated equation (called the *weak* or *variational* form) is exactly what the finite element method discretises, and whether it exists or not is determined by the bilinear form that comes from it. The Lax-Milgram theorem turns two simple conditions on that bilinear form (continuity and coercivity) into existence and uniqueness of a weak solution. This transfer of derivatives is useful as it lets us look for solutions in Sobolev spaces instead of in spaces of smooth functions.

Let V be a Hilbert space. A bilinear form $a(\cdot, \cdot) : V \times V \rightarrow \mathbb{R}$ is *continuous* if there exists a constant $C > 0$ such that

$$|a(u, v)| \leq C \|u\|_V \|v\|_V \quad \forall u, v \in V, \quad (9)$$

and *coercive* if there exists a constant $\alpha > 0$ such that

$$a(u, u) \geq \alpha \|u\|_V^2 \quad \forall u \in V. \quad (10)$$

These two properties are the hypotheses of the following Lax-Milgram theorem [5, Theorem 4.36].

Theorem 2.12 (Lax-Milgram). *Let V be a Hilbert space and let $a(\cdot, \cdot) : V \times V \rightarrow \mathbb{R}$ be a continuous, coercive bilinear form. Then for every continuous linear functional $\ell \in V^*$ there exists a unique $u \in V$ such that*

$$a(u, v) = \ell(v) \quad \forall v \in V. \quad (11)$$

We use this existence and uniqueness later, where it makes the solution operator $S : m \mapsto u$ of Definition 4.3 well-defined, and hence the reduced functional we minimise.

3 The Forward Poisson Problem

With the function spaces, weak formulations, and the Lax-Milgram theorem now in place, we are ready to study the first of our two model problems, the Poisson equation. We look at this first as it is a relatively simple problem, one that is linear and elliptic, with no time dependence. This makes it the natural setting in which we can build up a PDE-constrained optimisation problem (the forward solve, the adjoint equation, the reduced gradient, and the numerical optimisation) one piece at a time, before we repeat the same programme for the harder, nonlinear Burgers equation in Section 7. It is also natural to begin here because the existence and uniqueness of a solution follow directly from the Lax-Milgram theorem we have just stated. In this section we take the first step, the *forward* problem: for a given control (here the source term m) we solve for the corresponding state u , leaving the optimisation over m to Section 4.

Our goal is to rewrite the PDE in a form that involves only first derivatives of the solution, so that we can look for solutions in $H_0^1(\Omega)$ rather than in spaces of classical (twice differentiable) solutions.

3.1 Strong formulation

Let m be a given source term (for example $m \in L^2(\Omega)$). The strong form of the forward problem is

$$-\Delta u = m \quad \text{in } \Omega, \quad u = 0 \quad \text{on } \partial\Omega. \quad (12)$$

3.2 Derivation of the weak form

We choose a test function in

$$v \in H_0^1(\Omega), \quad (13)$$

which vanishes on $\partial\Omega$ in the trace sense. Multiplying equation (12) by v and integrating over Ω gives us

$$\int_{\Omega} (-\Delta u) v \, dx = \int_{\Omega} m v \, dx. \quad (14)$$

The left-hand side contains second derivatives of u . To remove one of them, we recall Green's first identity [5, Lemma 5.23]:

$$\int_{\Omega} (-\Delta u) v \, dx = \int_{\Omega} \nabla u \cdot \nabla v \, dx - \int_{\partial\Omega} \frac{\partial u}{\partial n} v \, ds, \quad (15)$$

where n denotes the outward unit normal on $\partial\Omega$ and $\partial u / \partial n := \nabla u \cdot n$ is the normal derivative of u . Because $v \in H_0^1(\Omega)$ vanishes on $\partial\Omega$ (because of the boundary conditions), the boundary integral vanishes, and we obtain

$$\int_{\Omega} (-\Delta u) v \, dx = \int_{\Omega} \nabla u \cdot \nabla v \, dx. \quad (16)$$

Substituting this into equation (14) gives

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} m v \, dx, \quad (17)$$

and this identity must hold for all test functions $v \in H_0^1(\Omega)$. We collect the result into the following definition.

Definition 3.1 (Weak formulation of the forward Poisson problem). Let $m \in L^2(\Omega)$. The *weak formulation* of equation (12) is: find $u \in H_0^1(\Omega)$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} m v \, dx \quad \forall v \in H_0^1(\Omega). \quad (18)$$

It is also convenient to introduce the bilinear form

$$b(u, v) := \int_{\Omega} \nabla u \cdot \nabla v \, dx \quad (19)$$

and the linear functional

$$\ell(v) := \int_{\Omega} m v \, dx, \quad (20)$$

so that equation (18) now reads

$$\text{find } u \in H_0^1(\Omega) \text{ such that } b(u, v) = \ell(v) \quad \forall v \in H_0^1(\Omega). \quad (21)$$

The bilinear form b is continuous and, by the Poincaré inequality, coercive on $H_0^1(\Omega)$. The linear functional ℓ is bounded on $H_0^1(\Omega)$ whenever $m \in L^2(\Omega)$. The Lax-Milgram theorem then guarantees existence and uniqueness of the weak solution.

4 The Poisson Optimal Control Problem

Now, we have set up the forward problem and showed it is well posed, so each source term m determines the state u . We now reverse the question. Rather than taking the source as given, we ask which source term m makes the resulting state u best match a given desired state d , while penalising large controls. In practice it is often this reversed direction, achieving a desired outcome rather than predicting one, that we care about. As an example, if we let u be the steady-state temperature of a body whose boundary is held fixed and m be a distributed heat source, we are asking which heating m brings the temperature closest to a desired profile d without expending excessive power. Problems of this shape, in which an input to a PDE is tuned to steer its solution towards a target, are very common in inverse problems and optimal design. The Poisson optimal control problem is a PDE-constrained optimisation problem of the form introduced in Section 2.2 [13].

Throughout the Poisson sections, we set

$$V := H_0^1(\Omega), \quad M := L^2(\Omega). \quad (22)$$

4.1 Objective functional

Let $d \in L^2(\Omega)$ be a given desired state and let $\alpha > 0$.

Definition 4.1 (Poisson objective functional). The *objective functional* $J : V \times M \rightarrow \mathbb{R}$ is defined by

$$J(u, m) := \frac{1}{2} \|u - d\|_{L^2(\Omega)}^2 + \frac{\alpha}{2} \|m\|_M^2. \quad (23)$$

The first term measures the misfit between the state u and the desired state d . The second term penalises large controls; the parameter $\alpha > 0$, called the *Tikhonov* parameter, or equivalently the regularisation parameter, balances accuracy of fit against regularity of the control. Note that without the regularisation term the problem would be ill posed: the control-to-state map is smoothing, so many very different controls would produce nearly the same state and the misfit term alone fixes no unique, stable minimiser. The quadratic penalty makes the objective strictly convex and coercive in m , which ensures a unique minimiser that depends continuously on the data.

4.2 The constrained optimisation problem

Combining Definition 4.1 with the weak state equation equation (21), we get that the PDE-constrained optimisation problem reads as follows.

Definition 4.2 (Poisson PDE-constrained optimisation problem). The *Poisson PDE-constrained optimisation problem* is to find $(u, m) \in V \times M$ minimising

$$\min_{(u, m) \in V \times M} J(u, m) \quad (24)$$

subject to

$$b(u, v) = \ell(v) \quad \forall v \in V, \quad (25)$$

where the bilinear form b and the linear functional ℓ are those of equations (19) and (20).

4.3 Reduced formulation

The problem of Definition 4.2 is a *constrained* minimisation: u and m are not independent, but are instead tied together by the state equation, and such problems in $V \times M$ are awkward to attack with the gradient-based methods that we will use in the rest of this dissertation. However, since the state equation determines u uniquely from m , we have that the state is not a free unknown, and we can remove it, reducing the problem to an *unconstrained* minimisation over the control alone [13].

For each $m \in M$ there exists a unique state $u \in V$ satisfying the state equation, so we may view the state as a function of the control. To make this precise, we name the map that sends each control to its state as the solution operator:

Definition 4.3 (Poisson solution operator). The *solution operator* $S : M \rightarrow V$ assigns to each $m \in M$ the unique solution $S(m) := u(m) \in V$ of the weak state equation associated with m .

Using the solution operator we eliminate the state from the optimisation problem.

Definition 4.4 (Poisson reduced functional). The *reduced functional* $\tilde{J} : M \rightarrow \mathbb{R}$ is defined by

$$\tilde{J}(m) := J(S(m), m) = J(u(m), m). \quad (26)$$

The second equality is only a change of notation: $S(m)$ and $u(m)$ are two names for the same object, the unique state produced by the control m . The key step is that $\tilde{J}$ is a function of m alone, which is legitimate because the state equation determines that state uniquely, so that once m is fixed no independent choice of u remains.

We then have that the constrained problem of Definition 4.2 is equivalent to the unconstrained minimisation problem

$$\min_{m \in M} \tilde{J}(m). \quad (27)$$

This reduced functional is the central object of the rest of this work. It is what every optimiser we study minimises, its gradient (computed by the adjoint method of Section 5) is what the Riesz map acts on, and its Hessian is where the mesh-dependence of Section 10 appears.

5 Derivative of the Reduced Functional and the Adjoint Equation

We have now reduced the problem to the unconstrained minimisation of $\tilde{J}$ over the control space. We want to do this minimisation with a gradient-based method, the natural choice for optimising over a function space, but these methods need the derivative of $\tilde{J}$. Getting it by differentiating through the state equation would require a separate linearised solve for each control direction δm , whereas if we use the adjoint method, we can get the derivative in every direction from a single additional solve. It does this by introducing an auxiliary variable, the *adjoint*, whose governing equation is chosen precisely so that the unknown state variation cancels from the expression for the derivative; the derivative in every direction can then be obtained from one adjoint solution. We first carry out this derivation for a general steady problem, and only then apply it to the Poisson problem. The argument is the same for any problem of this shape, so deriving it once in general lets us reuse it for the Burgers problem later. Throughout we use the chain rule (Theorem 2.11) and the calculus of partial Fréchet derivatives (Definition 2.10) introduced in Section 2.5.

5.1 The general derivation

We work with a general steady problem. The state u lives in a Hilbert space V , the control m lives in a Hilbert space M , and the objective is a functional $J : V \times M \rightarrow \mathbb{R}$. Each control m determines a unique state $u(m) \in V$ through the state equation, and the reduced functional is $\tilde{J}(m) = J(u(m), m)$ as before.

Applying the chain rule to $\tilde{J}(m) = J(u(m), m)$ in the direction $\delta m \in M$ gives

$$d\tilde{J}(m; \delta m) = \partial_u J(u, m; du_m(m; \delta m)) + \partial_m J(u, m; \delta m), \quad (28)$$

where $du_m(m; \delta m)$ is the total derivative of the state with respect to the control in the direction δm [13, p. 32]. Writing

$$\delta u := du_m(m; \delta m) \in V, \quad (29)$$

this becomes

$$d\tilde{J}(m; \delta m) = \partial_u J(u, m; \delta u) + \partial_m J(u, m; \delta m). \quad (30)$$

This derivative involves two different objects: the objective J , through its partial derivatives, and the state equation, which is what determines the state variation δu . We now finish the derivation at this general level in three steps: we differentiate the constraint to get an equation for δu , we define an adjoint variable chosen precisely so that δu cancels from the derivative, and we read off the result.

To differentiate the state equation, we first need to package it as a single object with the state and the control as explicit arguments. The following definition does exactly this.

Definition 5.1 (Forward model operator). The *forward model operator* $F(u, m; v)$ is the residual of the weak state equation, linear in the test function v , such that for each control m , the state $u(m) \in V$ is the unique solution of

$$F(u(m), m; v) = 0 \quad \forall v \in V \quad (31)$$

[13, p. 29].

The first step is to obtain the equation that determines δu , which we get by differentiating the state equation. Equation (31) holds for every control m , and its left-hand side depends on m both directly and through the state $u(m)$, so differentiating in the direction δm gives the following equation.

Definition 5.2 (Tangent linear equation). The *tangent linear equation*, also known as the tangent linear model [13, p. 32], is

$$\partial_u F(u, m; \delta u, v) + \partial_m F(u, m; \delta m, v) = 0 \quad \forall v \in V. \quad (32)$$

Once the partial derivatives of F are known, this is a linear equation determining δu from δm .

The only obstacle to evaluating $d\tilde{J}$ cheaply is the term $\partial_u J(u, m; \delta u)$ in the chain rule, which drags in the direction-specific state variation δu . The adjoint is defined precisely to remove it.

Definition 5.3 (Adjoint variable). The *adjoint variable* $\lambda \in V$ is the unique solution of

$$\partial_u F(u, m; w, \lambda) = \partial_u J(u, m; w) \quad \forall w \in V. \quad (33)$$

Note where λ is in this equation. It occupies the test-function slot of $\partial_u F$, while the direction slot is filled by the arbitrary w . Compared with the tangent linear equation, the roles of the direction and the test function are swapped, and this swap is what the word *adjoint* refers to.

The adjoint is exactly what we need to remove δu from the chain rule. Choosing the direction $w = \delta u$ in the adjoint equation gives

$$\partial_u J(u, m; \delta u) = \partial_u F(u, m; \delta u, \lambda), \quad (34)$$

and choosing the test function $v = \lambda$ in the tangent linear equation (32) gives

$$\partial_u F(u, m; \delta u, \lambda) = -\partial_m F(u, m; \delta m, \lambda). \quad (35)$$

Substituting these into equation (30), the state variation δu disappears completely:

$$d\tilde{J}(m; \delta m) = \partial_m J(u, m; \delta m) - \partial_m F(u, m; \delta m, \lambda) \quad \forall \delta m \in M. \quad (36)$$

So now, the right-hand side only involves the control direction δm and the single adjoint solution λ , and hence one adjoint solve gives the derivative of $\tilde{J}$ in every direction.

One step of the recipe remains. The derivative $d\tilde{J}(m)$ is an element of the dual space M^* , but a gradient-based optimiser steps along directions in M itself, so we use the Riesz map of Definition 2.8 to convert the former into the latter. Which inner product the Riesz map belongs to is a genuine choice, and the consequences of that choice are the subject of Section 10. Every gradient in this dissertation, whether ℓ^2 , L^2 or H^1 , is an instance of the following definition.

Definition 5.4 (*H-gradient*). Let H be a choice of inner product on M , with Riesz map $\mathcal{R}_H$. The *H-gradient* of $\tilde{J}$ at m is

$$\nabla_H \tilde{J}(m) := \mathcal{R}_H(d\tilde{J}(m)) \in M. \quad (37)$$

This completes the general derivation. For a concrete problem, we compute the four partial derivatives, solve the adjoint equation, read the derivative off equation (36), and apply the Riesz map of our chosen inner product.

5.2 The Poisson tangent linear equation

We now apply the general derivation to the Poisson problem. Following the recipe above, all we have to do is compute the four partial derivatives, starting with those of the forward model operator, which we first use (Definition 5.1) in the Poisson problem.

Definition 5.5 (Poisson forward model operator). The *forward model operator* of the Poisson problem is defined as the residual of the weak state equation:

$$F(u, m; v) := b(u, v) - (m, v)_{L^2(\Omega)}. \quad (38)$$

The weak state equation (21) is then exactly $F(u(m), m; v) = 0$ for all $v \in V$, so the Poisson problem fits the general setting. We compute the two partial derivatives of the forward model operator F . First, we want the derivative with respect to u . For $\varepsilon \in \mathbb{R}$, set $u_\varepsilon := u + \varepsilon \delta u$. Using linearity of b in its first argument,

$$F(u_\varepsilon, m; v) = b(u, v) - (m, v)_{L^2(\Omega)} + \varepsilon b(\delta u, v). \quad (39)$$

The corresponding difference quotient is

$$\frac{F(u + \varepsilon \delta u, m; v) - F(u, m; v)}{\varepsilon} = b(\delta u, v), \quad (40)$$

and taking $\varepsilon \rightarrow 0$ gives

$$\partial_u F(u, m; \delta u, v) = b(\delta u, v). \quad (41)$$

Next, the derivative with respect to m . By the analogous argument, perturbing m in the direction δm , we get

$$\partial_m F(u, m; \delta m, v) = -(\delta m, v)_{L^2(\Omega)}. \quad (42)$$

Substituting equations (41) and (42) into the general tangent linear equation (32) gives its explicit form for the Poisson problem.

Proposition 5.6 (Poisson tangent linear equation). *The state variation $\delta u \in V$ satisfies*

$$b(\delta u, v) = (\delta m, v)_{L^2(\Omega)} \quad \forall v \in V. \quad (43)$$

Notice that the tangent linear equation (43) has exactly the same form as the forward weak problem equation (21), with δu and δm in place of u and m . This is because the Poisson problem is linear, and a linear problem is its own tangent linear model.

5.3 Partial derivatives of the Poisson objective functional

Now we compute the partial derivatives of the objective functional J . First, the derivative of J with respect to u . Setting $u_\varepsilon := u + \varepsilon v$ and expanding, we get

$$J(u_\varepsilon, m) = J(u, m) + \varepsilon(u - d, v)_{L^2(\Omega)} + \frac{\varepsilon^2}{2} \|v\|_{L^2(\Omega)}^2, \quad (44)$$

and so

$$\partial_u J(u, m; v) = (u - d, v)_{L^2(\Omega)}. \quad (45)$$

Next, the derivative of J with respect to m . Setting $m_\varepsilon := m + \varepsilon \delta m$ and expanding $\|m + \varepsilon \delta m\|^2$,

$$J(u, m_\varepsilon) = J(u, m) + \alpha \varepsilon (m, \delta m)_{L^2(\Omega)} + \frac{\alpha \varepsilon^2}{2} \|\delta m\|_{L^2(\Omega)}^2, \quad (46)$$

giving

$$\partial_m J(u, m; \delta m) = \alpha (m, \delta m)_{L^2(\Omega)}. \quad (47)$$

5.4 The Poisson adjoint problem

The adjoint equation reverses the direction of the state equation. Where the state equation maps the control forward to the state, the adjoint equation propagates the sensitivity of the objective backward, to the control [13, p. 33]. Substituting the Poisson partial derivatives equations (41) and (45) into the general definition of the adjoint variable (Definition 5.3) gives $b(w, \lambda) = (u - d, w)_{L^2(\Omega)}$ for all $w \in V$. Since b is symmetric (we have $b(u, v) = b(v, u)$ because $\nabla u \cdot \nabla v = \nabla v \cdot \nabla u$), this gives the adjoint equation for the Poisson problem in explicit form.

Proposition 5.7 (Poisson adjoint equation). *The adjoint variable $\lambda \in V$ satisfies*

$$b(\lambda, v) = (u - d, v)_{L^2(\Omega)} \quad \forall v \in V. \quad (48)$$

The *adjoint problem* is to find the $\lambda \in V$ that satisfies this adjoint equation. It is a single Poisson-type solve, with the same bilinear form b as the forward problem of Definition 3.1, but driven by the misfit $u - d$ in place of the control m . The bilinear form b is continuous and coercive on V , and the right-hand side is a continuous linear functional on V , so existence and uniqueness of the adjoint follow from Lax-Milgram [13, p. 14]. Its one solution λ is all we need to evaluate the derivative of $\tilde{J}$ in every direction, which is what we do next.

5.5 Derivative of the Poisson reduced functional

With all four partial derivatives computed, the derivative of $\tilde{J}$ falls straight out of the general formula equation (36). Substituting the Poisson partials $\partial_m J(u, m; \delta m) = \alpha(m, \delta m)_{L^2(\Omega)}$ and $\partial_m F(u, m; \delta m, \lambda) = -(\delta m, \lambda)_{L^2(\Omega)}$ gives

$$d\tilde{J}(m; \delta m) = (\lambda, \delta m)_{L^2(\Omega)} + \alpha(m, \delta m)_{L^2(\Omega)} \quad \forall \delta m \in M, \quad (49)$$

where $u = u(m)$ is the state corresponding to m and λ is the adjoint solution.

5.6 The L^2 -gradient

We now have the derivative $d\tilde{J}$, but a gradient-based method cannot use it as it stands. These methods improve the control by stepping along a search direction, so they need an actual element of the control space M to add to the current control m , whereas the derivative is a functional that acts on directions rather than being one itself. So, we need to convert it into a genuine direction in M , and it is in this conversion that the choice of inner product matters.

The reduced derivative equation (49) is a continuous linear functional acting on directions $\delta m \in M$, which means that it is an element of the dual space M^* . By the Riesz representation theorem (Theorem 2.7), there exists a unique element of M representing it. The following is the H -gradient of Definition 5.4, with H taken to be the L^2 inner product.

Definition 5.8 (L^2 -gradient). Let $\mathcal{R}_{L^2} : M^* \rightarrow M$ denote the Riesz map associated with the $L^2(\Omega)$ inner product. The L^2 -gradient of $\tilde{J}$ at m is defined by

$$\nabla \tilde{J}(m) := \mathcal{R}_{L^2}(d\tilde{J}(m)) \in M, \quad (50)$$

that is, $\nabla \tilde{J}(m)$ is the unique element of M satisfying

$$d\tilde{J}(m; \delta m) = (\nabla \tilde{J}(m), \delta m)_{L^2(\Omega)} \quad \forall \delta m \in M. \quad (51)$$

Linearity of the inner product lets us factor equation (49) as

$$d\tilde{J}(m; \delta m) = (\lambda + \alpha m, \delta m)_{L^2(\Omega)} \quad \forall \delta m \in M. \quad (52)$$

Comparing with the definition of $\nabla \tilde{J}$ gives us that

$$\nabla \tilde{J}(m) = \lambda + \alpha m. \quad (53)$$

This completes the gradient derivation for the Poisson problem. A single forward solve for the state u and a single adjoint solve for λ combine to give $\nabla \tilde{J}(m) = \lambda + \alpha m$, the direction a gradient-based optimiser uses at each step.

6 Numerical Implementation and Verification of the Poisson Problem

Having derived the weak form of the forward problem, the adjoint equation and the L^2 -gradient of the reduced functional, we now implement the Poisson optimal control problem numerically. The implementation serves two purposes: to verify that the finite element discretisation of the forward problem behaves as expected, and to confirm that the adjoint-based gradient computation is correct before extending the same approach to more complicated problems in Sections 9 and 10.

We carry out the numerical work in two stages. First, we implement the forward Poisson problem in plain Python using a simple P_1 finite element method on a triangular mesh; the hand-written implementation makes the main finite element steps explicit, namely mesh construction, basis functions, element-wise assembly and the imposition of homogeneous Dirichlet boundary conditions. Second, we implement the same forward problem in Firedrake [11] together with the adjoint solve, the gradient computation and the Taylor test. The two implementations share the same mathematical content; the first makes the finite element method explicit, and the second expresses it more compactly using a domain-specific language.

6.1 Forward Poisson by hand

We start from scratch, without a finite element library, so that nothing about the method is hidden. Implementing the forward solve by hand puts every step (the mesh, the basis functions, the assembly and the boundary conditions) in plain view, where each can be checked directly against the weak form of Section 3.

Before describing that implementation, we define three finite element notions on which it rests: the mesh it is built on, the degrees of freedom it solves for, and the numerical quadrature it uses to evaluate the element integrals.

Definition 6.1 (Mesh). A *mesh*, or *triangulation*, $\mathcal{T}_h$ of a polygonal domain Ω is a finite set of triangles $\{K_i\}_{i=1}^N$ such that the interiors of distinct triangles are disjoint, the union of the triangles is the closure of Ω , and no vertex of one triangle lies in the interior of an edge of another [5, Defn. 1.2]. The last condition makes the mesh *conforming*, meaning that it has no hanging nodes. Each $K \in \mathcal{T}_h$ is a *cell*, or equivalently an *element*, and we write h_K for its diameter and

$$h := \max_{K \in \mathcal{T}_h} h_K \quad (54)$$

for the *mesh size*. A mesh is *uniform* when all of its cells have the same diameter, and *non-uniform*, or *graded*, when they do not.

Definition 6.2 (Degree of freedom). A *degree of freedom* is one of the unknowns in the finite element system, namely the coefficient multiplying a single basis function. For the P_1 Lagrange space used here, there is one degree of freedom per mesh vertex.

Definition 6.3 (Numerical quadrature). A *quadrature rule* approximates the integral of a function f over an element K by a weighted sum of its values at finitely many points,

$$\int_K f \, dx \approx \sum_q w_q f(x_q), \quad (55)$$

where the x_q are the *quadrature points* and the w_q the corresponding *weights*. The rule has *degree* p if this approximation is exact whenever f is a polynomial of degree at most p .

We implement the forward problem in plain Python using a continuous piecewise linear finite element method. The domain $\Omega = (0, 1)^2$ is triangulated using a structured mesh obtained by splitting each square cell into two triangles (Figure 1), and the discrete space is the standard P_1 Lagrange space.

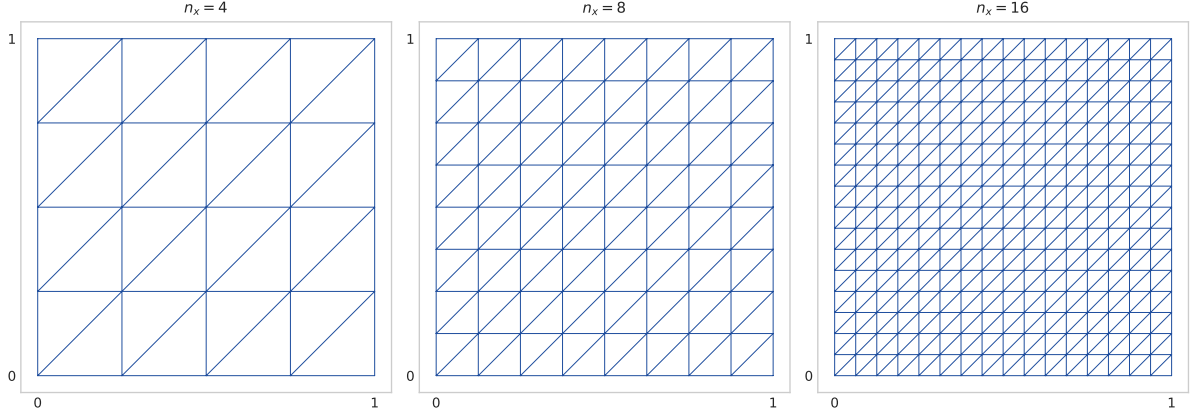


Figure 1: The structured triangulation of the unit square $\Omega = (0, 1)^2$ we used for the forward Poisson problem, shown at three refinement levels $n_x = 4, 8, 16$. Each square cell is split into two triangles along its bottom-left to top-right diagonal, giving $2n_x^2$ elements and $(n_x + 1)^2$ vertices. The convergence study refines this mesh uniformly up to $n_x = 512$. The hand-written implementation and Firedrake’s `UnitSquareMesh` produce this identical mesh.

We assemble the global linear system one element at a time. For each triangle, we first map it onto a reference triangle and apply numerical quadrature (Definition 6.3), which gives us a local stiffness matrix and a local load vector. We then add these local contributions into a single global sparse system. Finally, since the problem has homogeneous Dirichlet boundary conditions, we modify the assembled system so that the boundary degrees of freedom (Definition 6.2) are fixed to zero. These steps are summarised in Algorithm 1.

Algorithm 1 Assembly and solution of the P_1 forward Poisson system, as carried out by hand.

- 1: construct the structured triangular mesh of Ω
 - 2: initialise the global stiffness matrix $A \leftarrow 0$ and load vector $b \leftarrow 0$
 - 3: **for all** triangles K with vertices x_0, x_1, x_2 **do**
 - 4: form the affine map to the reference triangle, $J \leftarrow [x_1 - x_0 \ x_2 - x_0]$
 - 5: $\text{area}(K) \leftarrow \frac{1}{2} |\det J|$
 - 6: compute the physical basis gradients $\nabla \phi_i \leftarrow J^{-\top} \nabla_{\xi} \hat{\phi}_i$
 - 7: local stiffness $A_{ij}^K \leftarrow \text{area}(K) \nabla \phi_i \cdot \nabla \phi_j$
 - 8: local load $b_i^K \leftarrow \sum_q w_q |\det J| m(x_q) \hat{\phi}_i(\xi_q)$ ▷ quadrature
 - 9: add A^K and b^K into the global A and b
 - 10: **end for**
 - 11: **for all** boundary vertices p **do** ▷ homogeneous Dirichlet condition
 - 12: overwrite row p of A with the identity row and set $b_p \leftarrow 0$
 - 13: **end for**
 - 14: solve the linear system $Au = b$
-

The hand-written implementation focuses on clarity rather than efficiency. Writing the assembly explicitly makes the connection between the weak form derived in Section 3 and the finite element matrices that appear in the computation transparent.

6.2 Forward Poisson solve in Firedrake

The hand-written solve is transparent, but it only does the forward problem. Everything that follows, the adjoint solve, the gradient, and the mesh-dependence experiments, relies on the algorithmic differentiation that Firedrake and its pyadjoint extension provide, so we now want to implement the same forward problem there. This also gives an independent check, since two

different codes should now agree on the same discretisation.

So, we now implement the same forward problem in Firedrake [11] using continuous Galerkin elements of degree one. The variational forms are written directly from the weak formulation equation (21), and the homogeneous Dirichlet boundary condition is imposed strongly on $\partial\Omega$. The complete forward solve is shown in Figure 2.

```
V = FunctionSpace(mesh, "CG", 1)
u = TrialFunction(V)
v = TestFunction(V)

a = inner(grad(u), grad(v)) * dx
L = m * v * dx
bc = DirichletBC(V, 0.0, "on_boundary")

u_h = Function(V)
solve(a == L, u_h, bcs=bc)
```

Figure 2: The forward Poisson solve in Firedrake. The bilinear form a and linear form L are transcribed almost verbatim from the weak formulation equation (21), with m the source term interpolated onto the finite element space.

The Firedrake implementation is notably more concise than the plain Python version, but represents the same finite element problem. Working in Firedrake also lets us express the forward solve, the adjoint solve and the gradient computation in a form that remains close to the mathematical notation introduced earlier in the work; this is critical for the adjoint-based experiments of Section 10, which rely on the algorithmic differentiation provided by the pyadjoint extension to Firedrake [9].

6.3 Manufactured solution and convergence study

To test the correctness of the forward implementations, we use the method of manufactured solutions, following the presentation in Sect. 6.3 of Ham and Cotter [5]. We prescribe a smooth function satisfying the homogeneous boundary conditions and choose the source term accordingly. We define

$$u_{\text{exact}}(x, y) = \sin(\pi x) \sin(\pi y), \quad (56)$$

which vanishes on $\partial\Omega$. The corresponding source term is

$$m(x, y) = -\Delta u_{\text{exact}}(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y). \quad (57)$$

With this choice, the continuous problem admits the known exact solution $u = u_{\text{exact}}$.

Let u_h denote the finite element approximation on a mesh of size h . For sufficiently smooth solutions, standard finite element theory predicts that the $L^2(\Omega)$ error for piecewise linear elements satisfies

$$\|u - u_h\|_{L^2(\Omega)} = \mathcal{O}(h^2). \quad (58)$$

We estimate the experimental order of convergence between two successive mesh levels by

$$q = \frac{\log(e_H/e_h)}{\log(h_H/h_h)}, \quad (59)$$

where $e_h = \|u - u_h\|_{L^2(\Omega)}$.

The errors and rates observed on the two implementations, the plain Python P_1 assembler and Firedrake on the same sequence of uniformly refined meshes, are shown side by side in Tables 1a and 1b.

n_x	h	L^2 error	rate
4	0.2500	$7.5963 \cdot 10^{-2}$	-
8	0.1250	$2.0409 \cdot 10^{-2}$	1.8961
16	0.0625	$5.2009 \cdot 10^{-3}$	1.9724
32	0.0312	$1.3066 \cdot 10^{-3}$	1.9930
64	0.0156	$3.2704 \cdot 10^{-4}$	1.9982
128	0.0078	$8.1786 \cdot 10^{-5}$	1.9996
256	0.0039	$2.0448 \cdot 10^{-5}$	1.9999
512	0.0020	$5.1121 \cdot 10^{-6}$	2.0000

(a) Hand-written P_1 implementation.

N	h	L^2 error	rate
4	0.2500	$6.2103 \cdot 10^{-2}$	-
8	0.1250	$1.8332 \cdot 10^{-2}$	1.7603
16	0.0625	$4.7854 \cdot 10^{-3}$	1.9376
32	0.0312	$1.2095 \cdot 10^{-3}$	1.9842
64	0.0156	$3.0321 \cdot 10^{-4}$	1.9960
128	0.0078	$7.5855 \cdot 10^{-5}$	1.9990
256	0.0039	$1.8967 \cdot 10^{-5}$	1.9998
512	0.0020	$4.7420 \cdot 10^{-6}$	1.9999

(b) Firedrake implementation.

Table 1: Manufactured-solution convergence tests for the forward Poisson problem, for (a) the hand-written P_1 assembler and (b) Firedrake, on the same uniformly refined meshes. In both implementations the observed rates approach the expected value of 2, giving confidence that the forward discretisation is correct.

Both sets of errors are plotted against the mesh size on log-log axes in Figure 3.

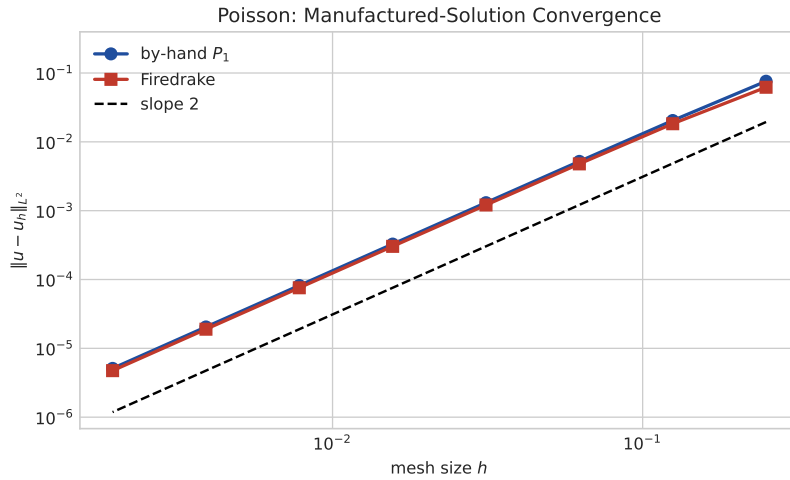


Figure 3: Manufactured-solution convergence of the two forward Poisson implementations on log-log axes (the data of Tables 1a and 1b). Note that as the two implementations lie almost on top of one another, the slope-2 reference is drawn offset below them, so that its slope can be compared with the data.

This confirms the expected $\mathcal{O}(h^2)$ rate, since we can see that the data run parallel to the offset slope-2 line. The two implementations still nearly coincide with each other, so it is hard to tell them apart. Rescaling the error by h^2 shows the same information more clearly, since a second-order method then appears as a flat plateau instead of a diagonal, and the two implementations separate into distinct curves, as in Figure 4.

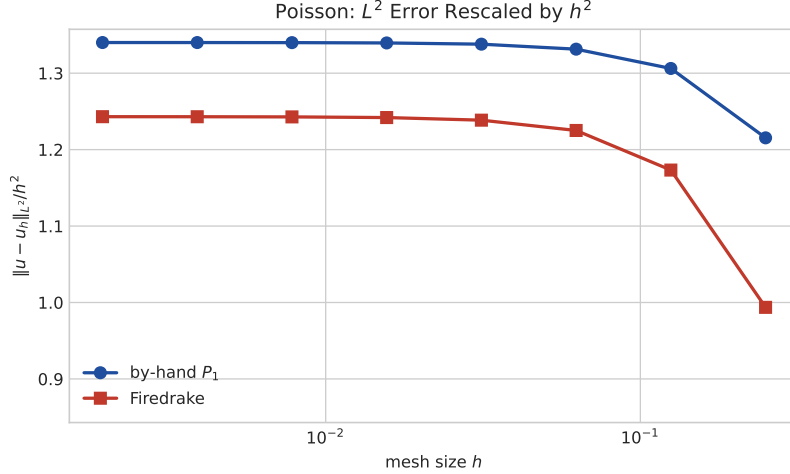


Figure 4: Manufactured-solution convergence of the two forward Poisson implementations (the data of Tables 1a and 1b), with the L^2 error rescaled by h^2 . Each curve plateaus to a constant as h decreases, which is the signature of the $\mathcal{O}(h^2)$ rate for P_1 elements; the roll-off at coarse h on the right is the pre-asymptotic regime, where the rate has not yet reached two. The by-hand plateau sits a little above Firedrake’s for the reason discussed below.

Both implementations recover the theoretical second-order rate, confirming that the forward discretisation is correct. The two do not report the same absolute error. The hand-written code is consistently a little larger, and its rescaled error in Figure 4 plateaus near 1.34 against Firedrake’s 1.24. This gap is a quadrature effect, not a difference in the discretisation, since both solve the same P_1 problem on the same mesh. The hand-written assembler evaluates the load vector and the L^2 error with a fixed degree-two quadrature rule, whereas Firedrake selects a higher quadrature degree for the smooth manufactured source $2\pi^2 \sin(\pi x) \sin(\pi y)$ and for the error integrand. The coarser rule integrates these slightly less accurately and so reports a marginally larger error. A degree-two rule is still exact enough to preserve the $\mathcal{O}(h^2)$ rate, which is why the two curves are parallel rather than divergent.

6.4 Adjoint solve and gradient computation

We now implement the gradient in Firedrake. This is the step that matters most for the rest of the work, as every optimiser we will later study uses this gradient at each iteration, so it must be computed correctly.

The derivation of Section 5 states that the gradient of the reduced functional is computed by first solving the adjoint equation and then forming the L^2 -gradient $\nabla \tilde{J}(m) = \lambda + \alpha m$. The Firedrake implementation does this in three stages:

1. solve the forward Poisson problem to obtain the state u ;
2. solve the adjoint problem to obtain λ ;
3. compute the gradient as the function $\lambda + \alpha m$.

The numerical implementation mirrors the theoretical derivation. In particular, it exploits the fact that the adjoint equation lets us compute the derivative of the reduced functional without solving a separate tangent linear problem for every perturbation direction.

6.5 Taylor test for gradient verification

After implementing the adjoint and gradient in Firedrake, we carry out a Taylor test to verify that the computed gradient is correct. The Taylor test is a standard practical check for adjoint implementations [9].

Let δm be a perturbation direction and let $h > 0$ be a small parameter. The reduced functional satisfies the Taylor expansion

$$\tilde{J}(m + h\delta m) = \tilde{J}(m) + h \, \mathrm{d}\tilde{J}(m; \delta m) + \mathcal{O}(h^2). \quad (60)$$

If the gradient implementation is correct, the second-order remainder

$$R_2(h) := \left| \tilde{J}(m + h\delta m) - \tilde{J}(m) - h \, \mathrm{d}\tilde{J}(m; \delta m) \right| \quad (61)$$

decays as $\mathcal{O}(h^2)$ as $h \rightarrow 0$.

Using a smooth perturbation direction, we obtained the remainder sequence reported in Table 2.

h	second-order remainder	observed rate
$1.000 \cdot 10^{-1}$	$1.8923 \cdot 10^{-6}$	-
$5.000 \cdot 10^{-2}$	$4.7307 \cdot 10^{-7}$	2.0000
$2.500 \cdot 10^{-2}$	$1.1827 \cdot 10^{-7}$	2.0000
$1.250 \cdot 10^{-2}$	$2.9567 \cdot 10^{-8}$	2.0000
$6.250 \cdot 10^{-3}$	$7.3917 \cdot 10^{-9}$	2.0000

Table 2: Taylor test for the gradient of the reduced Poisson functional. The observed second-order decay of the Taylor remainder is strong evidence that the adjoint solve and the gradient computation are implemented correctly.

7 The Forward Problem for the Scalar Viscous Burgers Equation

Having done the full programme (forward solve, adjoint equation, reduced gradient, and numerical verification) through for the linear Poisson problem, we now repeat it for a harder, nonlinear model problem: the scalar viscous Burgers equation. Following the notation established earlier, we write the state variable as u and the control as m . This is a scalar simplification of the time-dependent viscous Burgers model considered by Schwedes et al. [13], whose treatment is vector-valued. The main new features compared with the Poisson problem are nonlinearity in u (through the convection term), time dependence (which we deal with using backward Euler in Section 9), and a backward-in-time adjoint solve.

7.1 Strong formulation

Let $\Omega \subset \mathbb{R}$ be a bounded spatial domain and let $T > 0$. The scalar viscous Burgers equation reads

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2} = m \quad \text{in } (0, T) \times \Omega, \quad (62)$$

subject to homogeneous Dirichlet boundary conditions

$$u = 0 \quad \text{on } (0, T) \times \partial\Omega, \quad (63)$$

and initial condition

$$u(0, x) = u_0(x) \quad \text{for } x \in \Omega. \quad (64)$$

Here, $\nu > 0$ is the viscosity parameter.

7.2 Weak formulation

For each fixed $t \in (0, T)$, we take the spatial state space to be $V := H_0^1(\Omega)$. Let $v \in V$ be a test function. Multiplying equation (62) by v and integrating over Ω gives

$$\int_{\Omega} \frac{\partial u}{\partial t} v \, dx + \int_{\Omega} u \frac{\partial u}{\partial x} v \, dx - \nu \int_{\Omega} \frac{\partial^2 u}{\partial x^2} v \, dx = \int_{\Omega} m v \, dx. \quad (65)$$

Integrating the diffusion term by parts and using $v = 0$ on $\partial\Omega$,

$$\int_{\Omega} \frac{\partial u}{\partial t} v \, dx + \int_{\Omega} u \frac{\partial u}{\partial x} v \, dx + \nu \int_{\Omega} \frac{\partial u}{\partial x} \frac{\partial v}{\partial x} \, dx = \int_{\Omega} m v \, dx \quad \forall v \in V, \quad (66)$$

for almost every $t \in (0, T)$.

The convection term $u \partial_x u$ makes the Burgers dynamics qualitatively different from the linear Poisson problem. The local wave speed is the solution value u itself, so taller parts of the profile travel faster than shorter ones. This carries the profile forward, and the faster-moving regions of the solution catch up with the slower regions ahead of them, so the front of the wave steepens. The viscosity ν prevents this front from becoming a true discontinuity, and it also slowly damps the amplitude. Figure 5 shows this behaviour for a representative forward solve. This nonlinear steepening is one of the two main new features of the Burgers case; the other is the time dependence, which is what makes the adjoint run backward in time (Section 8).

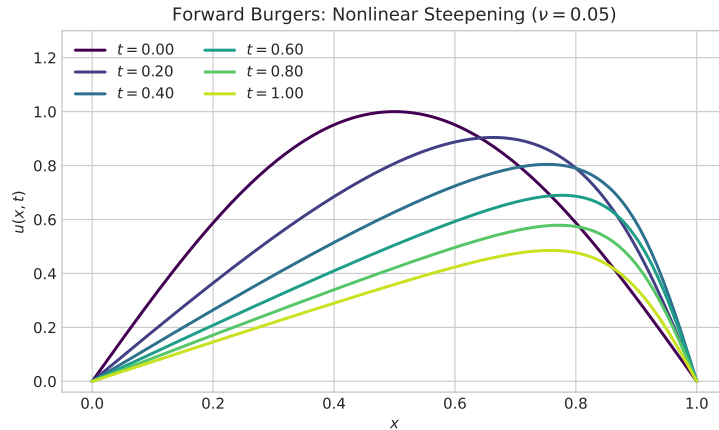


Figure 5: A representative solution of the forward Burgers equation with viscosity $\nu = 0.05$, starting from $u_0(x) = \sin(\pi x)$ with no forcing, shown at successive times.

8 Derivative of the Reduced Functional for the Burgers Problem

With the forward Burgers problem of Section 7 now in place, we extend the adjoint-based derivative analysis of Section 5 to the scalar Burgers problem. Here we have to be careful about which problem we differentiate. In practice, we never solve a time-dependent problem over the whole of space-time at once, since doing so would turn a problem in d space dimensions into one in $d + 1$ dimensions, which is far too expensive. Instead, the standard approach, and the one our implementation takes (Section 9), is to discretise in time first; we use the backward Euler method. The forward problem then becomes a *sequence* of spatial problems, one per time step, solved in turn, and the functional we minimise is a functional of these time-discrete states. We therefore differentiate this time-discrete problem directly, so that the derivation matches what we actually compute. The forward map is a composition of time steps, so we differentiate it using the chain rule together with the implicit function theorem, applying the

general derivation of Section 5.1 one step at a time, while keeping space continuous throughout. The spatial discretisation then proceeds exactly as for the Poisson problem. The adjoint of the time-continuous problem still appears, but only at the end, as the limit of the discrete adjoint as $\Delta t \rightarrow 0$.

8.1 Time discretisation and the composed forward problem

Let $0 = t_0 < t_1 < \dots < t_N = T$ be a uniform grid on $[0, T]$ with step $\Delta t = T/N$, and write $u^n \in V$ for the approximation to $u(t_n, \cdot)$, with $V := H_0^1(\Omega)$ as before and the initial state $u^0 = u_0$ is given and fixed. The backward Euler discretisation of the weak form equation (66) replaces the time derivative by a difference quotient, and evaluates every other term at the new time level. Given u^n , the next state $u^{n+1} \in V$ solves $F^{n+1}(u^{n+1}, u^n, m^{n+1}; v) = 0$ for all $v \in V$, where

$$F^{n+1}(u^{n+1}, u^n, m^{n+1}; v) := \int_{\Omega} \frac{u^{n+1} - u^n}{\Delta t} v + u^{n+1} \frac{\partial u^{n+1}}{\partial x} v + \nu \frac{\partial u^{n+1}}{\partial x} \frac{\partial v}{\partial x} - m^{n+1} v \, dx, \quad (67)$$

and $m^{n+1} \in L^2(\Omega)$ is the control at the new time level. This is one residual per time step, each of exactly the form assumed in the general setting of Section 5.1, except that every residual now involves *two* states: the one being solved for, u^{n+1} , and the already-known previous state u^n .

Each step is a nonlinear spatial problem (solved by Newton's method in Section 9), and by the implicit function theorem it defines the new state as a differentiable function of the data it depends on: $u^{n+1} = S^{n+1}(u^n, m^{n+1})$. Chaining the steps together, the forward map is a composition moving forwards in time,

$$u^1 = S^1(u^0, m^1), \quad u^2 = S^2(u^1, m^2), \quad \dots, \quad u^N = S^N(u^{N-1}, m^N), \quad (68)$$

so each state u^n depends on the controls $m^1, \dots, m^n$ up to its own time level and on nothing later.

The control is the collection $m = (m^1, \dots, m^N)$, living in the product space $M_{\Delta t} := L^2(\Omega)^N$, and the objective is the time-discrete tracking functional

$$J(u, m) := \Delta t \sum_{n=1}^N \left(\frac{1}{2} \|u^n - d^n\|_{L^2(\Omega)}^2 + \frac{\alpha}{2} \|m^n\|_{L^2(\Omega)}^2 \right), \quad d^n := d(t_n, \cdot), \quad (69)$$

which is the backward Euler quadrature of the space-time tracking objective, and the objective the implementation of Section 9 evaluates. On $M_{\Delta t}$ we use the matching time-discrete L^2 inner product

$$(m, m')_{\Delta t} := \Delta t \sum_{n=1}^N (m^n, m'^n)_{L^2(\Omega)}. \quad (70)$$

The reduced functional is $\tilde{J}(m) := J(u(m), m)$ as before, where $u(m) = (u^1(m), \dots, u^N(m))$ is the sequence of states produced by the composition above.

The derivation now follows the same three steps as the general case: differentiate each constraint to obtain a tangent linear system, introduce one adjoint variable per constraint, chosen so that every state variation cancels at once, and read the gradient off the result. The two new features are the nonlinearity of the convection term, which we have to linearise, and the coupling between time levels, which is what makes the adjoint run backward in time.

8.2 The tangent linear system

As in the Poisson case (Section 5.2), the first step is to write the state variations in terms of the control perturbation by differentiating the constraints, of which there are now N , one per time step. Let $\delta m = (\delta m^1, \dots, \delta m^N)$ be a control perturbation and let δu^n denote the resulting variation in u^n . Since the initial state is fixed, $\delta u^0 = 0$. Each constraint

$$F^{n+1}(u^{n+1}(m), u^n(m), m^{n+1}; v) = 0 \quad (71)$$

holds identically in m , so differentiating in the direction δm gives, for $n = 0, \dots, N-1$,

$$\partial_{u^{n+1}} F^{n+1}(\delta u^{n+1}, v) + \partial_{u^n} F^{n+1}(\delta u^n, v) + \partial_{m^{n+1}} F^{n+1}(\delta m^{n+1}, v) = 0 \quad \forall v \in V, \quad (72)$$

where we omit from the notation the point (u^{n+1}, u^n, m^{n+1}) at which the partial derivatives are evaluated, to keep the formulas short; each derivative is still taken at these states. Each residual has three arguments, so there are three partial derivatives per step, and the only nonlinear term is the convection term $u^{n+1} \frac{\partial u^{n+1}}{\partial x}$ of equation (67). The derivative of the nonlinear convection term is

$$(u + \varepsilon \delta u) \frac{\partial}{\partial x} (u + \varepsilon \delta u) = u \frac{\partial u}{\partial x} + \varepsilon \left(\delta u \frac{\partial u}{\partial x} + u \frac{\partial \delta u}{\partial x} \right) + \mathcal{O}(\varepsilon^2). \quad (73)$$

We now substitute $u^{n+1} + \varepsilon w$ for u^{n+1} in the step residual equation (67). The time and diffusion terms are linear in u^{n+1} , the source term does not contain it, and the convection term expands according to equation (73), applied with $u = u^{n+1}$ and $\delta u = w$. Collecting the terms by powers of ε gives

$$\begin{aligned} F^{n+1}(u^{n+1} + \varepsilon w, u^n, m^{n+1}; v) &= F^{n+1}(u^{n+1}, u^n, m^{n+1}; v) \\ &\quad + \varepsilon \int_{\Omega} \frac{w v}{\Delta t} + \left(w \frac{\partial u^{n+1}}{\partial x} + u^{n+1} \frac{\partial w}{\partial x} \right) v + \nu \frac{\partial w}{\partial x} \frac{\partial v}{\partial x} dx + \mathcal{O}(\varepsilon^2). \end{aligned} \quad (74)$$

As in the Poisson case, the difference quotient converges as $\varepsilon \rightarrow 0$ to the ε -coefficient, so the derivative of the step residual with respect to u^{n+1} in a direction $w \in V$ is

$$\partial_{u^{n+1}} F^{n+1}(w, v) = \int_{\Omega} \frac{w v}{\Delta t} + \left(w \frac{\partial u^{n+1}}{\partial x} + u^{n+1} \frac{\partial w}{\partial x} \right) v + \nu \frac{\partial w}{\partial x} \frac{\partial v}{\partial x} dx, \quad (75)$$

while the previous state and the control only enter the residual linearly, giving

$$\partial_{u^n} F^{n+1}(w, v) = - \int_{\Omega} \frac{w v}{\Delta t} dx, \quad \partial_{m^{n+1}} F^{n+1}(\delta m^{n+1}, v) = - \int_{\Omega} \delta m^{n+1} v dx. \quad (76)$$

Starting from $\delta u^0 = 0$, the system equation (72) is therefore a forward sweep, where each step is a *linear* spatial problem determining δu^{n+1} from δu^n and δm^{n+1} . Evaluating the derivative of $\tilde{J}$ this way would cost one such sweep for every control direction δm , which is exactly the cost the adjoint avoids.

8.3 The adjoint system

As in the Poisson case (Section 5.4), we introduce adjoint variables to remove the state variations from the derivative, but now there are N of them, one per constraint. The chain rule, applied in each of the $2N$ arguments of the objective equation (69), gives

$$d\tilde{J}(m; \delta m) = \sum_{k=1}^N \left(\partial_{u^k} J(\delta u^k) + \partial_{m^k} J(\delta m^k) \right), \quad (77)$$

with the partial derivatives computed exactly as in the Poisson case (Section 5.3): only the k -th summand of equation (69) contains u^k or m^k , and differentiating that summand gives

$$\partial_{u^k} J(w) = \Delta t (u^k - d^k, w)_{L^2(\Omega)}, \quad \partial_{m^k} J(\delta m^k) = \alpha \Delta t (m^k, \delta m^k)_{L^2(\Omega)}, \quad (78)$$

and the terms that we have to be aware of are again the ones containing the state variations δu^k . We define the adjoint variables so that all of them cancel at once.

Definition 8.1 (Discrete Burgers adjoint variables). The adjoint variables $\lambda^N, \lambda^{N-1}, \dots, \lambda^1 \in V$ are the solutions, in this order, of

$$\partial_{u^k} F^k(w, \lambda^k) + \partial_{u^k} F^{k+1}(w, \lambda^{k+1}) = \partial_{u^k} J(w) \quad \forall w \in V, \quad k = N, N-1, \dots, 1, \quad (79)$$

where, so that the $k = N$ equation takes the same form as the others, we set

$$\lambda^{N+1} := 0. \quad (80)$$

To see how equation (79) arises, recall the general adjoint equation of Definition 5.3:

$$\partial_u F(u, m; w, \lambda) = \partial_u J(u, m; w) \quad \forall w \in V. \quad (81)$$

Substituting the k -th constraint for F , so that the state is u^k and the adjoint is λ^k , gives

$$\partial_{u^k} F^k(w, \lambda^k) = \partial_{u^k} J(w) \quad \forall w \in V, \quad (82)$$

which is equation (79) without its second term. In Section 5.1 this was the whole equation, because the state appeared in exactly one constraint. Here the state u^k appears not only in its own step residual F^k , but also as the previous state in the next one, F^{k+1} , so both residuals contribute a term to the equation for λ^k ; adding the contribution of F^{k+1} , tested against its own adjoint λ^{k+1} , gives the extra term of equation (79). This extra term only involves λ^{k+1} , which is already known when we solve for λ^k . As a result, the system is solved backward in time, and equation (80) is the zero terminal condition that starts the backward sweep. Each equation uses the same linearised operator as the corresponding tangent step, with the roles of the direction and the test function swapped, so it is solvable whenever the linearised forward step is.

The cancellation works just as in the general case, with one extra step to tidy up the sums. We test the n -th tangent linear equation equation (72) with $v = \lambda^{n+1}$, and sum over $n = 0, \dots, N-1$. We then relabel by $k = n + 1$. Since $\delta u^0 = 0$, the term at the bottom of the middle sum drops out, and since $\lambda^{N+1} = 0$, we may append a vanishing term at its top. This gives

$$\sum_{k=1}^N \left(\partial_{u^k} F^k(\delta u^k, \lambda^k) + \partial_{u^k} F^{k+1}(\delta u^k, \lambda^{k+1}) \right) = - \sum_{k=1}^N \partial_{m^k} F^k(\delta m^k, \lambda^k). \quad (83)$$

Choosing $w = \delta u^k$ in the k -th adjoint equation equation (79) and summing over k , the left-hand side above is exactly $\sum_k \partial_{u^k} J(\delta u^k)$, and so

$$\sum_{k=1}^N \partial_{u^k} J(\delta u^k) = - \sum_{k=1}^N \partial_{m^k} F^k(\delta m^k, \lambda^k). \quad (84)$$

Every state variation is gone, at the cost of one backward sweep of adjoint solves, rather than one forward sweep of tangent solves per control direction.

Substituting the partial derivatives equations (75), (76) and (78) into equation (79) gives the explicit adjoint step: for $k = N, \dots, 1$, find $\lambda^k \in V$ such that

$$\int_{\Omega} \frac{\lambda^k - \lambda^{k+1}}{\Delta t} w + \left(w \frac{\partial u^k}{\partial x} + u^k \frac{\partial w}{\partial x} \right) \lambda^k + \nu \frac{\partial w}{\partial x} \frac{\partial \lambda^k}{\partial x} dx = \Delta t \int_{\Omega} (u^k - d^k) w dx \quad \forall w \in V. \quad (85)$$

Each adjoint step is a *linear* spatial problem for λ^k , even though the forward step was nonlinear. It is linearised about the computed forward state u^k , which must therefore be available, so the forward problem is solved and stored first. This is precisely the structure implemented in Section 9, namely a nonlinear forward sweep, then a sequence of linear solves backward in time from a zero terminal condition.

Finally, we check consistency with the time-continuous adjoint. Rescaling $\mu^k := \lambda^k / \Delta t$, substituting $\lambda^k = \Delta t \mu^k$ into equation (85) and dividing through by Δt gives

$$\int_{\Omega} \frac{\mu^k - \mu^{k+1}}{\Delta t} w + \left(w \frac{\partial u^k}{\partial x} + u^k \frac{\partial w}{\partial x} \right) \mu^k + \nu \frac{\partial w}{\partial x} \frac{\partial \mu^k}{\partial x} dx = \int_{\Omega} (u^k - d^k) w dx \quad \forall w \in V. \quad (86)$$

Now let $\Delta t \rightarrow 0$, with μ^k approximating $\mu(t_k)$. Since μ^{k+1} lives at the *later* time t_{k+1} , the difference quotient $(\mu^k - \mu^{k+1})/\Delta t$ converges to $-\partial\mu/\partial t$, and the remaining terms converge to their time-continuous counterparts, giving

$$\int_{\Omega} -\frac{\partial \mu}{\partial t} w + \left(w \frac{\partial u}{\partial x} + u \frac{\partial w}{\partial x} \right) \mu + \nu \frac{\partial w}{\partial x} \frac{\partial \mu}{\partial x} dx = \int_{\Omega} (u - d) w dx \quad \forall w \in V. \quad (87)$$

Integrating the second part of the convection contribution by parts in space, the boundary term vanishes because $w = 0$ on $\partial\Omega$, and

$$\int_{\Omega} \left(w \frac{\partial u}{\partial x} + u \frac{\partial w}{\partial x} \right) \mu dx = \int_{\Omega} w \frac{\partial u}{\partial x} \mu dx - \int_{\Omega} \frac{\partial}{\partial x} (u\mu) w dx = - \int_{\Omega} u \frac{\partial \mu}{\partial x} w dx. \quad (88)$$

Integrating the diffusion term by parts in the same way moves every spatial derivative off w , and the identity becomes $\int_{\Omega} (-\partial_t \mu - u \partial_x \mu - \nu \partial_{xx} \mu - (u - d)) w dx = 0$ for every $w \in V$, provided μ is smooth enough for the second derivative to exist. An integral that vanishes against *every* test function forces the bracket itself to vanish, since if the bracket were nonzero on some small region, a w concentrated on that region would make the integral nonzero [5, Sect. 5.3]. So μ satisfies the backward-in-time equation

$$-\frac{\partial \mu}{\partial t} - u \frac{\partial \mu}{\partial x} - \nu \frac{\partial^2 \mu}{\partial x^2} = u - d \quad \text{in } (0, T) \times \Omega, \quad (89)$$

with $\mu = 0$ on $(0, T) \times \partial\Omega$ and $\mu(T, x) = 0$. This is the adjoint obtained by deriving at the time-continuous level first. The discrete adjoint is therefore consistent with it, but it is the sequence equation (85), not equation (89), that the implementation solves, and it is the sequence that yields the exact derivative of the time-discrete functional we actually minimise.

8.4 Derivative of the reduced functional and the gradient

Substituting the cancellation equation (84) into the chain rule turns the derivative into the general formula equation (36), applied at each time level and summed. Using $\partial_{m^k} J(\delta m^k) = \alpha \Delta t (m^k, \delta m^k)_{L^2(\Omega)}$ and $\partial_{m^k} F^k(\delta m^k, \lambda^k) = -(\delta m^k, \lambda^k)_{L^2(\Omega)}$, this gives

$$d\tilde{J}(m; \delta m) = \sum_{k=1}^N \left(\partial_{m^k} J(\delta m^k) - \partial_{m^k} F^k(\delta m^k, \lambda^k) \right) = \sum_{k=1}^N (\lambda^k + \alpha \Delta t m^k, \delta m^k)_{L^2(\Omega)} \quad (90)$$

for all $\delta m \in M_{\Delta t}$.

To read off the gradient we apply the Riesz map of our chosen control inner product equation (70); the result is the H -gradient of Definition 5.4 for this choice of H . The gradient $g = (g^1, \dots, g^N)$ is the element of $M_{\Delta t}$ satisfying $d\tilde{J}(m; \delta m) = (g, \delta m)_{\Delta t} = \Delta t \sum_k (g^k, \delta m^k)_{L^2(\Omega)}$. Comparing with equation (90) level by level gives

$$g^k = \alpha m^k + \frac{\lambda^k}{\Delta t}, \quad k = 1, \dots, N. \quad (91)$$

This is exactly the gradient formula implemented in Section 9.3, including the factor $1/\Delta t$, which the time-continuous derivation does not produce. The reason is that the discrete adjoint λ^k carries one factor of Δt relative to the continuous adjoint, since it is $\mu^k = \lambda^k/\Delta t$ that converges as $\Delta t \rightarrow 0$. The Riesz map of the Δt -weighted inner product then removes that factor again. In the rescaled variable the gradient reads $g^k = \alpha m^k + \mu^k$, which recovers the same formal structure as the Poisson gradient equation (53), one time level at a time.

9 Numerical Implementation and Verification of the Burgers Problem

Having derived the weak formulation, the tangent linear and adjoint equations and the L^2 -gradient of the reduced functional for the scalar viscous Burgers equation, we now implement the corresponding optimal control problem numerically. The aim is the same as in Section 6, namely verifying that the discretisation of the forward problem behaves as expected, and checking that the adjoint-based gradient computation is correct.

In contrast to what we did with the Poisson problem, where we provided both a plain Python implementation and a Firedrake implementation, we treat the Burgers problem at the lower level only. The main new difficulties specific to Burgers' equation are the nonlinearity, the time dependence and the backward-in-time adjoint solve. Implementing these ingredients directly in plain Python makes the structure of the method more transparent before moving to a higher-level framework.

9.1 Scalar Burgers numerically

We consider the one-dimensional spatial domain $\Omega = (0, 1)$ and a final time $T > 0$. The implementation discretises the scalar viscous Burgers equation in space using continuous piecewise linear finite elements on a uniform mesh, and in time using the backward Euler method of Section 8.1, on a uniform grid of N_t steps of size $\Delta t = T/N_t$.

At each implicit time level, the nonlinear problem is solved using Newton's method. If U^n denotes the finite element coefficient vector at time t_n , then the fully discrete forward problem at t_{n+1} has the form

$$M_h \frac{U^{n+1} - U^n}{\Delta t} + C(U^{n+1}) + \nu K_h U^{n+1} = b^{n+1}, \quad (92)$$

where M_h is the finite element mass matrix, K_h is the stiffness matrix, $C(U^{n+1})$ is the nonlinear convection contribution, and b^{n+1} is the load vector associated with the source term or control at time t_{n+1} .

As with the plain Python Poisson implementation, this code prioritises clarity over efficiency. Writing the assembly, the time stepping and the Newton linearisation explicitly makes the path from the nonlinear weak form to the algebraic system solved at each time step explicit. The implementation imposes homogeneous Dirichlet boundary conditions strongly at the two endpoints of the interval.

9.2 Manufactured solution and convergence study

To verify the forward implementation, we again use the method of manufactured solutions. We prescribe a smooth exact solution satisfying the homogeneous Dirichlet boundary conditions, and choose the corresponding source term so that this exact solution satisfies the Burgers equation.

We define

$$u_{\text{exact}}(t, x) = e^{-t} \sin(\pi x), \quad (93)$$

which vanishes at $x = 0$ and $x = 1$ and therefore satisfies the boundary conditions. Substituting into the strong form equation (62) gives the manufactured source term

$$m_{\text{exact}}(t, x) = -e^{-t} \sin(\pi x) + \pi e^{-2t} \sin(\pi x) \cos(\pi x) + \nu \pi^2 e^{-t} \sin(\pi x). \quad (94)$$

We solve the numerical problem on a sequence of uniformly refined meshes with $N \in \{10, 20, 40, 80\}$ elements. Since the problem is time-dependent, the temporal discretisation is refined at the same rate by taking $N_t = 8N$, so that Δt remains proportional to the mesh size h .

We record two error measures: the final-time $L^2(\Omega)$ error,

$$e_h^{\text{final}} := \|u_h(T) - u_{\text{exact}}(T)\|_{L^2(\Omega)}, \quad (95)$$

and a fully discrete space-time error,

$$e_h^{\text{st}} := \left(\Delta t \sum_{n=1}^{N_t} \|u_h^n - u_{\text{exact}}(t_n)\|_{L^2(\Omega)}^2 \right)^{1/2}. \quad (96)$$

The experimental order of convergence between two successive refinements is

$$q = \frac{\log(e_H/e_h)}{\log(h_H/h_h)}. \quad (97)$$

The observed results are shown in Table 3.

N	N_t	h	e_h^{final}	rate	e_h^{st}	rate
10	80	0.1000	$5.840 \cdot 10^{-3}$	-	$1.838 \cdot 10^{-3}$	-
20	160	0.0500	$1.452 \cdot 10^{-3}$	2.0077	$4.590 \cdot 10^{-4}$	2.0027
40	320	0.0250	$3.577 \cdot 10^{-4}$	2.0197	$1.140 \cdot 10^{-4}$	2.0098
80	640	0.0125	$8.679 \cdot 10^{-5}$	2.0414	$2.810 \cdot 10^{-5}$	2.0209

Table 3: Manufactured-solution convergence test for the scalar Burgers forward solver. Both error measures decrease at an observed rate very close to 2, consistent with the expected behaviour of the fully discrete scheme for this smooth manufactured solution.

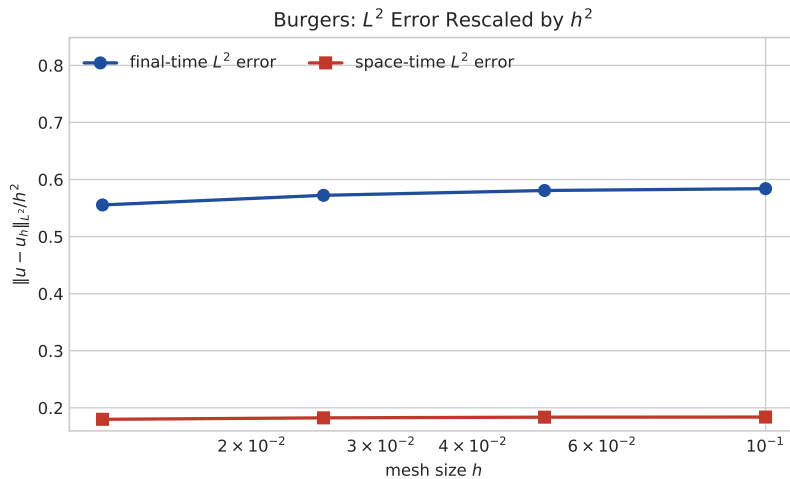


Figure 6: Manufactured-solution convergence of the forward Burgers solver (the data of Table 3), with each L^2 error rescaled by h^2 . Both error measures plateau to a constant, confirming the $\mathcal{O}(h^2)$ rate; the reason the rate is second order despite the first-order backward Euler scheme is discussed below.

Although the backward Euler discretisation is only first order in time, the observed second-order rate is governed by the P_1 spatial discretisation. Taking $N_t = 8N$ makes the time step $\Delta t = Th/8$ small relative to the mesh size, so the $\mathcal{O}(\Delta t)$ temporal error remains well below the $\mathcal{O}(h^2)$ spatial error over the resolutions tested. Refining the time step alone, on a fixed fine mesh, recovers the expected first-order temporal rate, and refining the mesh alone, on a fixed fine time grid, the second-order spatial rate.

9.3 Discrete adjoint solve and gradient computation

We implement the discrete adjoint equation and compute the gradient of the reduced functional at the fully discrete level, consistent with the backward Euler time discretisation of the forward problem.

If D^n denotes the discrete desired state at time t_n and m^n the discrete control, the fully discrete tracking functional, the mass-matrix representation of the objective equation (69), is

$$J_h(U, m) = \frac{\Delta t}{2} \sum_{n=1}^{N_t} \|U^n - D^n\|_{M_h}^2 + \frac{\alpha \Delta t}{2} \sum_{n=1}^{N_t} \|m^n\|_{M_h}^2, \quad (98)$$

where $\|v\|_{M_h}^2 := v^T M_h v$ is the mass-matrix representation of the $L^2(\Omega)$ norm.

We generate the desired state by first choosing a smooth reference control, solving the forward Burgers problem for that control, and taking the resulting discrete state trajectory as the target trajectory. We then choose a different trial control and solve the forward problem again in order to compute the state, the objective value, the adjoint and the gradient.

The adjoint solve follows the structure derived in Section 8.3. The nonlinear forward problem is solved first and its states stored, since the adjoint is linearised about them, and the adjoint systems equation (85), restricted to the finite element space, are then solved backward in time from the zero terminal condition [13, Sect. 1.4.5].

The resulting discrete gradient has the form

$$g^n = \alpha m^n + \frac{1}{\Delta t} p^n, \quad (99)$$

where p^n is the discrete adjoint variable at time level t_n . This is exactly the gradient formula equation (91) derived in Section 8.4, with p^n the spatial discretisation of λ^n .

For the representative trial control used in the experiments, the computed objective value was

$$J_h(m) = 1.323574 \times 10^{-4}. \quad (100)$$

This is the baseline value that the Taylor test of Section 9.4 differences against, and it sets the scale for reading the remainders of Table 4. The average number of Newton iterations required in the forward solve was approximately 3 per time step, indicating that the nonlinear solver converged reliably for the chosen problem parameters.

9.4 Taylor test for gradient verification

After implementing the discrete adjoint and gradient, we carry out a Taylor test in the same spirit as in Section 6.5.

Let δm be a perturbation direction in the discrete control space and let $h > 0$. The reduced functional satisfies

$$\tilde{J}_h(m + h\delta m) = \tilde{J}_h(m) + h d\tilde{J}_h(m; \delta m) + \mathcal{O}(h^2), \quad (101)$$

and if the gradient implementation is correct, the second-order remainder

$$R_2(h) := \left| \tilde{J}_h(m + h\delta m) - \tilde{J}_h(m) - h \, \mathrm{d}\tilde{J}_h(m; \delta m) \right| \quad (102)$$

should decay as $\mathcal{O}(h^2)$ as $h \rightarrow 0$.

Using a smooth perturbation direction normalised in the discrete control inner product, we obtained the results of Table 4.

h	second-order remainder	observed rate
$1.000 \cdot 10^{-1}$	$6.7317 \cdot 10^{-5}$	-
$5.000 \cdot 10^{-2}$	$1.6828 \cdot 10^{-5}$	2.0001
$2.500 \cdot 10^{-2}$	$4.2068 \cdot 10^{-6}$	2.0001
$1.250 \cdot 10^{-2}$	$1.0517 \cdot 10^{-6}$	2.0000
$6.250 \cdot 10^{-3}$	$2.6291 \cdot 10^{-7}$	2.0000

Table 4: Taylor test for the discrete reduced functional in the scalar Burgers problem. The observed second-order decay of the Taylor remainder is strong evidence that the discrete adjoint solve and gradient computation are implemented correctly.

Together, the verifications of Sections 6 and 9 show that the adjoint-based optimisation framework developed in Sections 5 and 8 extends successfully from a linear, stationary problem to a nonlinear, time-dependent one, from the simple Poisson example to the broader questions of PDE-constrained optimisation and mesh dependence taken up next in Section 10.

10 Mesh Dependence in PDE-Constrained Optimisation

So far, we have derived, discretised and verified the adjoint-based gradient for both the Poisson and the scalar viscous Burgers optimal control problems. In each case, the L^2 -gradient $\nabla \tilde{J}(m) = \lambda + \alpha m$ was identified using the Riesz representation theorem (Theorem 2.7). However, that identification is not unique, since it depends on the choice of Hilbert space structure used for the control space M . In this section we will look at the practical consequences of that choice when the reduced functional is minimised numerically. Following Schwedes et al. [13, Chap. 2], we show that two different but mathematically valid choices of inner product give optimisation algorithms whose iteration counts behave very differently as the mesh is refined.

10.1 The two Riesz maps

In Sections 2.4 and 5.6, we have already established that the Fréchet derivative $\mathrm{d}\tilde{J}(m)$ is an element of the dual space M^* , and that the Riesz map $\mathcal{R}_H : M^* \rightarrow M$ identifies each derivative with a primal element of M . Which primal element it returns depends on the inner product we give M , and there are two natural candidates: when $M = L^2(\Omega)$ is treated as an L^2 Hilbert space, the Riesz map $\mathcal{R}_{L^2}$ corresponds in the discrete setting to multiplication by the inverse of the Galerkin mass matrix [13, (1.125)]. When the same coefficient vector is taken instead as an element of $\mathbb{R}^d$ under the standard Euclidean (ℓ^2) inner product, no mass matrix appears and the Riesz map is the identity.

The two choices give us primal gradients that differ at every mesh refinement: the ℓ^2 representer carries the local cell sizes encoded in the diagonal of the mass matrix, while the L^2 representer does not. So, we can see that search directions and stopping criteria based on gradient norms behave differently in the two settings. It matters, then, which of the two an optimisation package uses internally, and whether the user can change it. Schwedes et al. [13, Sect. 2.1] note that

“many well-established continuous optimisation packages are not designed for function-based optimisation and are hard-coded to apply the Euclidean (ℓ^2) inner product”. That assessment dates from 2017 and is now only partly accurate, as we show later in this section.

10.2 Non-uniform meshes and the mesh ratio

Before stating the prediction we test, it is worth being precise about the meshes on which mesh dependence occurs at all, since what determines it turns out not to be how fine a mesh is, but how uneven it is.

The accuracy of a finite element solution is controlled by the size of each cell together with how rapidly the solution varies inside it [5, Sect. 3.4]. A uniform mesh must therefore be fine enough everywhere to resolve the most demanding part of the solution. However, that is expensive, since in two dimensions halving the cell size multiplies the number of cells, and with them the cost of every forward and adjoint solve, by four. Practical meshes are non-uniform instead, with small cells where the solution has fine structure and large cells where it is smooth. A solution may change sharply in a thin strip next to the boundary, or become arbitrarily steep at a corner that points into the domain, such as the inner corner of an L-shaped region. It may also carry a steep front that moves, as the Burgers solution of Section 7 does. Non-uniform meshes are therefore not an edge case constructed to stress test an optimiser, but instead what one would generally compute on in practice.

A concrete example makes the trade-off explicit. Consider the steady temperature in a cross-section of a room, heated by a small radiator against one wall and losing heat through a window set into the wall opposite. We model each as a narrow Gaussian, small compared with the room. The temperature is then nearly uniform across most of the room and varies sharply in two small regions, as the left panel of Figure 7 shows. This is the situation where a graded mesh is required, as the solution needs many cells at the radiator and the window, and almost none in between.

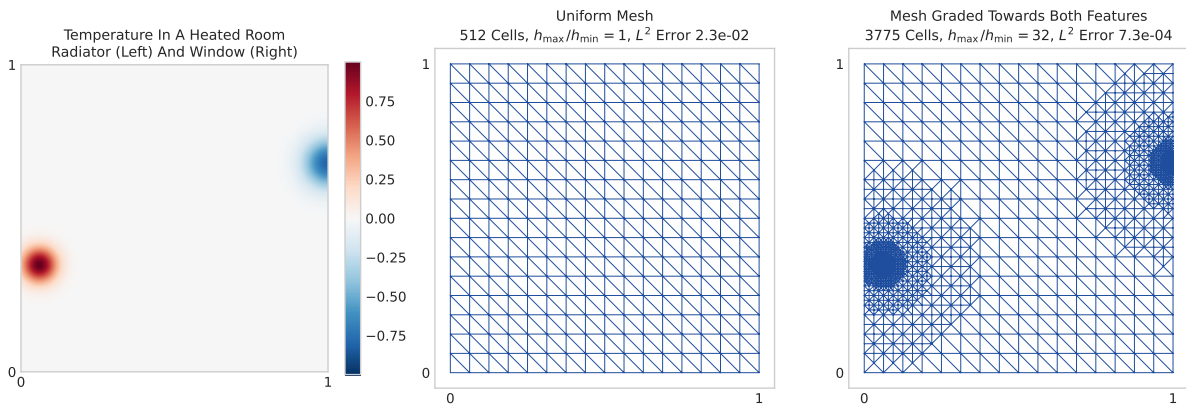


Figure 7: A room heated by a radiator and cooled through a window. The temperature field is nearly uniform across the room and varies sharply only at the two features (left). A uniform mesh spends its cells evenly and resolves neither feature well (centre). Grading the same background resolution towards both (right) reduces the L^2 error by a factor of about 31 for roughly seven times the cells, at the cost of a mesh ratio of 32.

To measure the uniformity of a mesh, recall the cell diameter h_K of Definition 6.1 and set

$$h_{\max} := \max_{K \in \mathcal{T}_h} h_K \quad \text{and} \quad h_{\min} := \min_{K \in \mathcal{T}_h} h_K \quad (103)$$

to be the diameters of the largest and smallest cells in the mesh. The *mesh ratio* $h_{\max}/h_{\min}$ equals one for a perfectly uniform mesh and grows as the grading becomes more severe, and we

will take it as a measure of the non-uniformity of the mesh. What matters for the argument that follows is that the ratio records only how unequal the cells are, and not how many of them there are. That distinction is the reason the mesh ratio, rather than the mesh size, indexes every experiment in this section.

We test the prediction of Schwedes et al. [13, Sect. 2.2], namely that, for a fixed gradient-norm convergence threshold, the iteration count of a quasi-Newton method depends on which Riesz map is used to interpret the derivative as a primal-space gradient. Concretely:

- (I) When the ℓ^2 Riesz map is used, the iteration count grows with the mesh ratio $h_{\max}/h_{\min}$ (mesh-dependent convergence).
- (II) When the L^2 Riesz map is used, the iteration count is bounded uniformly in $h_{\max}/h_{\min}$ (mesh-independent convergence).

10.3 The benchmark: Table 2.2 of Schwedes et al. [13]

The experiments of this section are a reproduction of Table 5, which is Table 2.2 of Schwedes et al. [13]. We reproduce it here so that it is clear exactly what we are comparing against.

inner product	implementation	$h_{\max}/h_{\min}$					
		4	8	16	32	64	128
ℓ^2	SciPy (BFGS)	27	47	75	97	133	189
ℓ^2	TAO (LMVM)	42	60	77	111	131	155
ℓ^2	IPOPT (Interior Point)	28	38	65	88	117	139
L^2	Moola (BFGS)	22	20	22	23	23	27

(a) $H = L^2(\Omega)$

inner product	implementation	$h_{\max}/h_{\min}$					
		4	8	16	32	64	128
ℓ^2	SciPy (BFGS)	25	53	118	222	244	594
ℓ^2	TAO (LMVM)	38	38	110	144	141	384
ℓ^2	IPOPT (Interior Point)	31	38	85	127	182	443
H^1	Moola (BFGS)	4	4	4	4	4	4

(b) $H = H^1(\Omega)$

Table 5: Reproduced from Table 2.2 of Schwedes et al. [13], showing iteration counts to reach $\|\mathcal{R}_H(J')\|_H \leq 10^{-7}$ for four optimisation packages on non-uniform meshes of increasing grading ratio $h_{\max}/h_{\min}$, under two choices of control-space inner product. The history length for the Hessian approximation is ten, except for TAO/LMVM where it is five. The three ℓ^2 rows grow with the mesh ratio, whereas the Moola row, which uses the function-space inner product, stays flat.

The columns sweep the grading ratio $h_{\max}/h_{\min}$ from 4 to 128. Each row is one optimisation package together with the inner product it uses internally, and the algorithm it runs is given in brackets. The four packages are the following.

- `scipy.optimize`, the optimisation module of SciPy, the general-purpose scientific Python library. Its row runs L-BFGS-B, the limited-memory, bound-constrained form of BFGS, the standard quasi-Newton method named after Broyden, Fletcher, Goldfarb and Shanno and described in Section 10.6.

- TAO, the Toolkit for Advanced Optimization, which is the optimisation component of PETSc, a widely used library for large-scale scientific computing [1]. Its row runs LMVM, short for limited-memory variable metric, which is TAO’s own form of limited-memory BFGS.
- IPOPT, the Interior Point OPTimizer, a solver for large-scale nonlinear problems. It runs an interior point method, a different family of algorithm from the quasi-Newton methods of the other three.
- Moola, a library written specifically for PDE-constrained optimisation. It also runs BFGS, but takes every inner product in the function-space norm through the Riesz map.

The first three rows, SciPy, TAO and IPOPT, all use the Euclidean ℓ^2 inner product of the coefficient vector, which is the default in general-purpose optimisation libraries. The last row, Moola, uses the function-space inner product, the L^2 norm in panel (a) and the H^1 norm in panel (b). The threshold $\varepsilon = 10^{-7}$ is applied to the gradient measured in that same norm, $\|\mathcal{R}_H(J')\|_H$, so that every row is stopped on a comparable quantity. Which algorithm a package runs is not what the table is testing, since the comparison being made is between the inner products. The pattern is the point of the whole section. The three ℓ^2 rows grow steadily as the mesh becomes more graded, while the Moola row stays essentially constant, around twenty in panel (a) and exactly four in panel (b).

Note that we will not reproduce every cell in this table. The scientific content is the contrast between the ℓ^2 rows and the function-space row, not the four packages individually. SciPy, TAO and IPOPT are three implementations of the same ℓ^2 choice, so one of them is enough to show the growth, and we take SciPy for that role (Tables 8 and 9). For the function-space row we write our own implementation, the Hilbert-space L-BFGS of Section 10.6, which takes every inner product in the chosen norm and so makes it clear where the metric enters. TAO is not discarded but given a study of its own (Sections 10.9 and 10.10 and Section 11), since whether it can be made mesh-independent is one of the contributions of this work. Finally, the table is a snapshot of the software as it stood in 2017, and the packages have since moved on. Our graded meshes are also built differently from theirs, concentrating the refinement in a fixed interior region [13, Sect. 2.3.1], so two meshes agreeing in $h_{\max}/h_{\min}$ need not give the same iteration count. Exact agreement is not to be expected in any case, since the counts depend on floating-point details, such as the order in which the linear algebra library accumulates a sum, that differ between machines and can decide on which iteration a run crosses the threshold. We therefore aim to reproduce the broad contrast between growth and flatness rather than the counts cell for cell. The TAO row in particular was run with a setting that Section 10.9 identifies as the reason it appears mesh-dependent.

10.4 Mesh generation

We use two families of non-uniform mesh. First, we use deterministic graded meshes because they are the meshes behind Table 2.2 of Schwedes et al. [13], whose refinement is fixed and whose $h_{\max}/h_{\min}$ ratio can be set to a prescribed value, which is what a faithful reproduction of the headline result requires. The randomly refined meshes come second, as a robustness check. They discard the smooth, structured grading of the first family and refine cells irregularly, so that any mesh-dependence we observe cannot be a side-effect of one tidy grading pattern; if the same behaviour survives on irregular meshes, it is a property of the inner product rather than of the mesh construction. We describe the two families in turn.

The deterministic graded meshes are built on a structured base. The unit square is meshed by Firedrake’s `UnitSquareMesh` function into an 8×8 grid of squares, each split into two triangles along a diagonal. A fixed interior sub-square $[0.4, 0.6]^2$ is then refined by repeated longest-edge

(Rivara) bisection [12]: in each round, every triangle whose centroid lies in the sub-square is bisected, and the bisection is propagated to neighbours so that the mesh stays conforming, with no hanging nodes. After $2\log_2 r$ rounds the cells inside the sub-square are a factor r finer than those outside, so the realised ratio $h_{\max}/h_{\min}$ is *exactly* the prescribed power of two rather than an approximation to it. Three of the six meshes are shown in Figure 8.

The cell counts and realised ratios of the six meshes, for $r \in \{4, 8, 16, 32, 64, 128\}$, are reported in Table 6.

target r	# cells	realised $h_{\max}/h_{\min}$
4	256	4.00
8	640	8.00
16	1 712	16.00
32	5 776	32.00
64	22 608	64.00
128	87 312	128.00

Table 6: The six deterministic graded meshes used in the experiments. The realised $h_{\max}/h_{\min}$ is an exact power of two by construction, and is measured from the realised cell diameters.

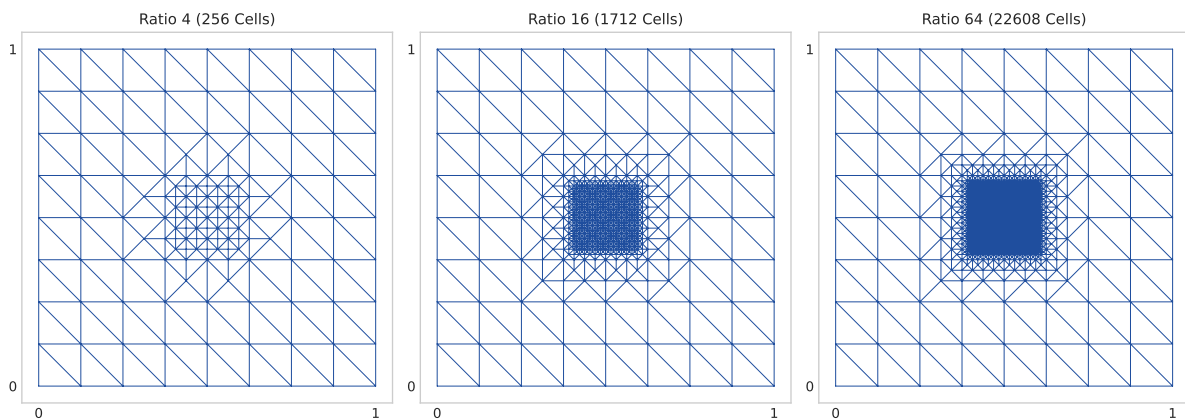


Figure 8: Three of the six deterministic graded meshes of Table 6, at ratios $r = 4, 16, 64$. The structured 8×8 base, squares split into triangles, is refined inside the central sub-square $[0.4, 0.6]^2$ by longest-edge bisection, so the smallest cells are a factor r finer than the outer ones. At $r = 64$ the refined region is so fine it appears almost solid.

The deterministic graded meshes above play the same role as the deterministically refined meshes of Schwedes et al. [13, Sect. 2.3.1], which are what are used in the Table 2.2 experiment reproduced in this section, though ours concentrate the refinement in a fixed interior sub-square rather than following their construction exactly, but this does not affect the results. We also implemented the random refinement scheme of Schwedes et al. [13, Sect. 2.3.2.1], in which each cell is independently marked with probability p and refined by longest-edge (Rivara) bisection [12], the refinement propagating to neighbours along the longest-edge propagation path so that the mesh stays conforming. However, applied over the whole domain, this random scheme cannot reach the high ratios of the graded meshes. The realised $h_{\max}/h_{\min}$ saturates near 5.7 even as the cell count grows, and lowering p only lifts the ceiling to roughly 8–11. The reason is that refinement spreads. Refining one cell eventually forces most of the domain to refine, so $h_{\max}$ shrinks almost as fast as $h_{\min}$. Restricting the marking to a fixed interior sub-square removes this ceiling and reaches the same range of ratios as the graded meshes, and the resulting interior random meshes are used for the corroborating experiments of Section 12.6. Their cell counts and

realised ratios are reported in Table 7, and three of them are shown in Figure 9. The realised ratios are exact powers of two even though the marking is random, because bisecting a right isosceles triangle along its hypotenuse produces two more triangles of the same shape. Every cell diameter therefore lies in a geometric sequence of common ratio $\sqrt{2}$, and the marking decides which cells reach which depth, and how many cells result, but not which sizes are available.

refinement level	# cells	realised $h_{\max}/h_{\min}$
4	314	4.00
6	772	8.00
8	1970	16.00
10	5450	32.00
12	16020	64.00
14	46998	128.00

Table 7: The six interior random-refined meshes used in the corroborating experiments of Section 12.6. At each level, cells whose centroid lies in the sub-square $(0.3, 0.7)^2$ are marked independently with probability $p = 0.35$ and refined by longest-edge bisection, from a fixed random seed so that the meshes are reproducible. Here $h_{\max}$ is a base cell outside the sub-square, which is never eligible for marking, and $h_{\min}$ is the deepest refined cell, so the ratio records how far the refinement has reached rather than a prescribed target. It spans the same range of 4 to 128 as the graded meshes of Table 6, so the two families are directly comparable.

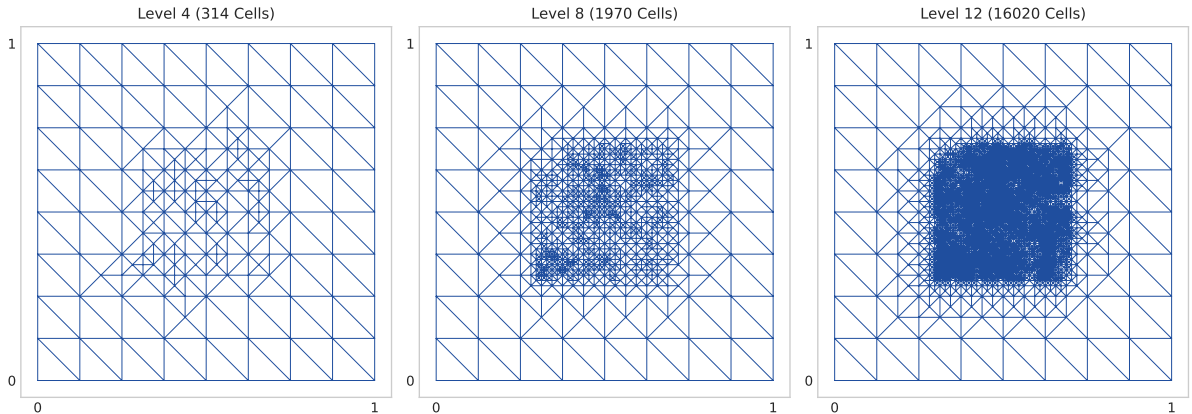


Figure 9: Three of the interior random-refined meshes used for the matching experiments of Section 12.6, at refinement levels 4, 8 and 12 (realised $h_{\max}/h_{\min} = 4, 16, 64$). Cells inside the sub-square $(0.3, 0.7)^2$ are marked at random and refined by longest-edge bisection, giving the irregular pattern seen here, in contrast to the smoothly graded meshes of Figure 8. The refinement is kept away from the domain boundary, so it never interacts with the Dirichlet conditions.

10.5 Implementation

Testing the hypothesis in code requires pinning down two things, namely which inner product each algorithm uses internally, and a stopping criterion applied identically to every algorithm regardless of that choice. We describe how each is handled.

The experiments are built on Firedrake [11] and its algorithmic-differentiation extension pyadjoint [9]. The choice of inner product on the control is made when the control is declared, by giving pyadjoint's `Control` a `riesz_map` of "L2", "L2" or "H1", which selects the discrete Riesz map applied when the dual derivative $d\tilde{J}(m)$ is converted into a primal gradient:


```
Jhat = ReducedFunctional(
    J, Control(m, riesz_map=riesz_map), tape=tape,
)
```

That single argument is the experimental variable of this section. The problem, the mesh, the forward solver and the stopping threshold are all held fixed, so any difference in iteration count between two runs must be due to it alone. The forward Poisson problem is solved by a direct LU factorisation, so the discrete gradient is computed to machine precision, and the convergence threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_H(J')\|_H$ is therefore never obscured by Krylov-solver noise.

Following Schwedes et al. [13, pp. 67–68], we apply the convergence criterion panel-wide rather than per algorithm, meaning that every algorithm in a panel is stopped on the same gradient norm $\|\mathcal{R}_H(J')\|_H \leq 10^{-7}$, regardless of the inner product it uses internally. This is what makes the rows comparable, since an algorithm allowed to stop on its own norm would be answering a different question from the one beside it. For an algorithm hard-coded to the ℓ^2 inner product, such as SciPy’s L-BFGS-B, the norm is therefore evaluated *outside* the algorithm, by disabling its own tolerances so that they never trigger and halting the run from an external check on $\|\mathcal{R}_H(J')\|_H$ at each iteration.

The problem can also be routed through PETSc’s TAO solver, which pyadjoint supports directly. We tried this first, since it is much the easiest route, but it did not reproduce the function-space row, and working out why turned into a study of its own. That is Section 10.9. The function-space rows of the experiments are produced instead by our own limited-memory BFGS, which follows Schwedes et al. [13, Sect. 1.5.4.1] and is stated in Section 10.6.

10.6 BFGS in a Hilbert space

The hypothesis of Section 10.2 is a claim about a single ingredient of a quasi-Newton method, namely the inner product it uses to turn a derivative into a search direction. Testing it convincingly needs an implementation where that ingredient is visible rather than buried, so that changing it is clearly the only thing that has changed. Neither package in Table 5 serves that purpose, since SciPy is hard-coded to ℓ^2 and TAO’s update runs inside a PETSc kernel that cannot readily be inspected from Python. So, we write the algorithm out ourselves, as a limited-memory BFGS where the metric is completely explicit, so that one can read off at every step where the inner product enters. This is the implementation that produces the function-space row of every experiment that follows.

We take H to be either $L^2(\Omega)$ or $H^1(\Omega)$, with the inner products

$$(a, b)_{L^2} = \int_{\Omega} a b \, dx, \quad (a, b)_{H^1} = \int_{\Omega} (a b + \nabla a \cdot \nabla b) \, dx. \quad (104)$$

Limited-memory BFGS never forms an approximate inverse Hessian explicitly. It keeps a short history of curvature pairs and applies the approximation to a vector through a two-loop recursion, which is no more than a sequence of inner products and vector updates. That structure is what makes the metric easy to locate, since the inner product appears in every one of those products and nowhere else, so replacing ℓ^2 by $(\cdot, \cdot)_H$ throughout is the entire modification.

At the current *iterate* $m_k \in M$, meaning the optimiser’s current approximation to the optimal control after k steps, let $g_k := \mathcal{R}_H(d\tilde{J}(m_k))$ denote the primal Riesz representer of the derivative. The algorithm stores the most recent $\ell \leq 5$ curvature pairs

$$s_j := m_{j+1} - m_j \in M, \quad y_j := g_{j+1} - g_j \in M, \quad \rho_j := \frac{1}{(s_j, y_j)_H}, \quad (105)$$

and forms the search direction p_k by the two-loop recursion

$$q \leftarrow g_k, \quad (106)$$

for $j = k - 1, k - 2, \dots, k - \ell$:

$$\alpha_j \leftarrow \rho_j(s_j, q)_H, \quad (107)$$

$$q \leftarrow q - \alpha_j y_j, \quad (108)$$

then

$$z \leftarrow \gamma_k q, \quad (109)$$

for $j = k - \ell, \dots, k - 1$:

$$\beta_j \leftarrow \rho_j(y_j, z)_H, \quad (110)$$

$$z \leftarrow z + (\alpha_j - \beta_j) s_j, \quad (111)$$

and finally

$$p_k \leftarrow -z, \quad (112)$$

in which every inner product is taken in $(\cdot, \cdot)_H$.

The recursion begins not from the identity but from an initial inverse Hessian $H_0 = \gamma_k I$, rescaled at each iteration by

$$\gamma_k = \frac{(s_{k-1}, y_{k-1})_H}{(y_{k-1}, y_{k-1})_H}, \quad (113)$$

the standard L-BFGS scaling of Schwedes et al. [13, Sect. 1.5.4.2], which adapts it to the local curvature. Omitting it, that is taking $\gamma_k = 1$, causes the algorithm to plateau several orders of magnitude above the threshold, even on a well-conditioned mesh.

The step length is chosen by a backtracking Armijo line search where, starting from $\alpha = 1$, the trial step is halved until the sufficient-decrease condition

$$\tilde{J}(m_k + \alpha p_k) \leq \tilde{J}(m_k) + c_1 \alpha (g_k, p_k)_H, \quad c_1 = 10^{-4}, \quad (114)$$

holds. Curvature pairs with $(s_j, y_j)_H \leq 0$ are discarded, and the iteration terminates once $\|g_k\|_H \leq \varepsilon = 10^{-7}$.

The Riesz representer g_k is obtained from the dual derivative by a direct LU factorisation of the inner-product matrix, M_h for L^2 and $M_h + K_h$ for H^1 . An iterative solve would be cheaper, but it would stop at a tolerance of its own, and whatever error it left behind would sit inside the very gradient norm that the convergence test measures. Since that test is applied at $\varepsilon = 10^{-7}$, a stalled optimiser would then be indistinguishable from an inaccurate Riesz map. The direct factorisation is accurate to machine precision, so no such ambiguity arises.

10.7 Numerical experiments: Panel (a), $H = L^2(\Omega)$

With both implementations in place, we can run the comparison the hypothesis asks for, changing the inner product and holding everything else fixed. We reproduce Panel (a) of Table 2.2 in Schwedes et al. [13], in which the control space is treated as the L^2 Hilbert space.

The state equation that we use is the weak Poisson problem equation (21), and the objective is equation (23) with target $d(x, y) = \sin(\pi x) \sin(\pi y)$ and Tikhonov parameter $\alpha = 10^{-4}$. The control is initialised at $m \equiv 0$, and every run is stopped at the threshold $\varepsilon = 10^{-7}$ on

$\|\mathcal{R}_{L^2}(J')\|_{L^2}$, with an L-BFGS history of five. Two algorithms are compared: SciPy’s L-BFGS-B [3, 14], which operates on the ℓ^2 coefficient vector, with the convergence test performed externally in L^2 ; and the Hilbert-space L-BFGS of Section 10.6 with $H = L^2(\Omega)$. The results are collected in Table 8 and plotted in Figure 11, and the recovered solution of the problem is shown in Figure 10.

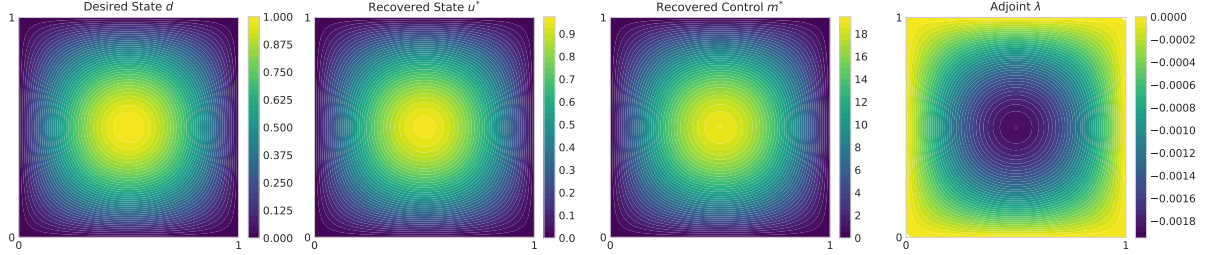


Figure 10: The Poisson optimal control problem of Panel (a), solved to optimality on a uniform 48×48 mesh. The recovered control m^* drives the state u^* to match the desired state d ; the adjoint λ is small because the state already tracks the target closely.

inner product	implementation	target $h_{\max}/h_{\min}$				
		4	8	16	32	64
ℓ^2	SciPy (L-BFGS-B, external check)	81	180	357	675	1490
L^2	Hilbert-space L-BFGS	6	6	6	6	6

Table 8: Iteration counts at convergence for the Poisson optimal control problem on the deterministic graded meshes of Table 6. Both rows used the same problem data, the same forward LU solver, the same convergence threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{L^2}(J')\|_{L^2}$ and the same L-BFGS history of five. The only difference between the rows is the inner product used internally by the algorithm.

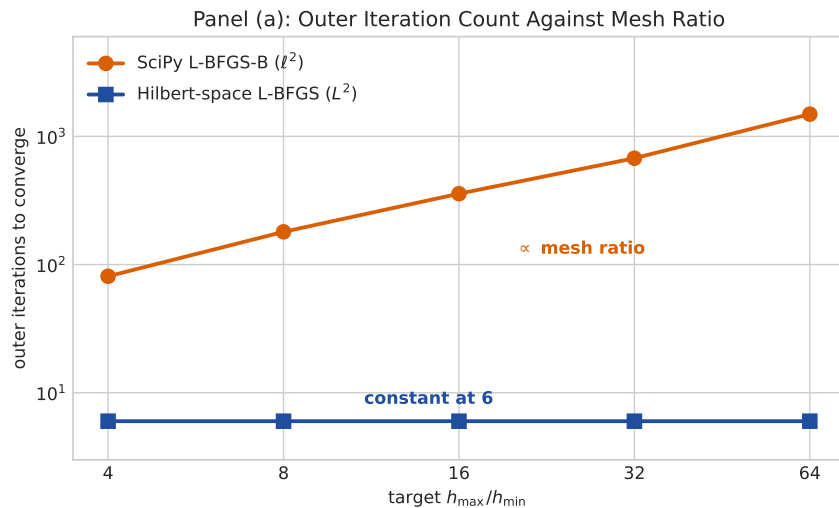


Figure 11: Iteration counts of Table 8 against the mesh ratio. On the ℓ^2 inner product the count doubles at every refinement; on the L^2 inner product it stays flat at six. The only difference between the two runs is the inner product used inside the optimiser.

The ℓ^2 row roughly doubles at every refinement (it increases by factors of 2.22, 1.98, 1.89 and 2.21), matching the polynomial-in- $h_{\max}/h_{\min}$ growth derived in Schwedes et al. [13, Sect. 2.2.4],

while the L^2 row is constant at six iterations across the whole sweep. This is the contrast predicted by hypotheses (I) and (II) of Section 10.2. Our two rows show the same broad pattern as the corresponding rows of Table 5, though the absolute counts differ because of implementation details, mainly the line search. Our backtracking Armijo search enforces only sufficient decrease, and so converges in fewer outer iterations than Moola’s strong-Wolfe search [4, 10], which additionally enforces a curvature condition.

10.8 Numerical experiments: Panel (b), $H = H^1(\Omega)$

Panel (a) fixed the inner product to L^2 . The hypothesis is a claim about the choice of inner product rather than about L^2 in particular, so it should still hold even if we replace that choice with another one. Panel (b) of Table 2.2 in Schwedes et al. [13] repeats the experiment with the H^1 inner product on the control space. Before setting the experiment up there is one thing we need to get right, and it makes a large difference to the result. The objective functional of Schwedes et al. [13, (1.219)] regularises in the *same* norm as the metric. It reads $\frac{1}{2}\|u - d\|_{L^2}^2 + \frac{\alpha}{2}\|m\|_M^2$ with $M = L^2$ or H^1 , so the norm appears twice, and changing M changes both. Moving from Panel (a) to Panel (b) therefore changes the regularisation term from $\frac{\alpha}{2} \int_{\Omega} m^2$ to

$$\frac{\alpha}{2}\|m\|_{H^1}^2 = \frac{\alpha}{2} \int_{\Omega} (m^2 + |\nabla m|^2) dx. \quad (115)$$

We pose this H^1 -regularised problem and, as in Panel (a), compare two algorithms stopped on the same threshold $\varepsilon = 10^{-7}$, now measured in the H^1 norm $\|\mathcal{R}_{H^1}(J')\|_{H^1}$, namely SciPy’s ℓ^2 L-BFGS-B, with the convergence test performed externally in H^1 , and the Hilbert-space L-BFGS of Section 10.6 with $H = H^1(\Omega)$. The results are given in Table 9.

inner product	implementation	target $h_{\max}/h_{\min}$					
		4	8	16	32	64	128
ℓ^2	SciPy (L-BFGS-B, external check)	44	59	107	188	451	1075
H^1	Hilbert-space L-BFGS	4	4	4	4	4	4

Table 9: Iteration counts at convergence for the H^1 -regularised Poisson optimal control problem on the deterministic graded meshes of Table 6, at the threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{H^1}(J')\|_{H^1}$. Both rows solve the same H^1 -regularised problem and reach the same discrete optimum; they differ only in the inner product used internally. The Hilbert-space L-BFGS row is mesh-independent at four iterations, reproducing the constant Moola row of Schwedes et al. [13, Table 2.2(b)] exactly.

In this table, we can see that the Hilbert-space L-BFGS row is flat at four iterations across the entire sweep, over which the realised mesh ratio grows by a factor of 32 and the number of cells by a factor of about 340. This reproduces the mesh-independent behaviour of the Moola row of Schwedes et al. [13] exactly, as their Table 2.2(b) also reports a constant four. The ℓ^2 row, by contrast, grows by roughly a factor of two at each refinement, in the same manner as the ℓ^2 row of Panel (a); the two algorithms reach the same discrete optimum and differ only in the inner product used internally.

We can measure what happens if the two norms are not taken to match. Applying the H^1 metric to the L^2 -regularised problem of Panel (a), that is, changing the inner product without changing the objective, the Hilbert-space L-BFGS still reaches the same L^2 -regularised optimum, but takes 100, 102, 103, 136, 113 and 141 iterations across the six meshes. That is more than an order of magnitude above the six iterations the same problem needs in Panel (a), where the metric and the regularisation agree. This mismatch, rather than any deficiency of the H^1 Riesz

map itself, is what produces those counts. When the metric matches the norm in which the problem is posed, the H^1 Riesz map delivers mesh-independent convergence, exactly as the L^2 Riesz map does in Panel (a).

10.9 The limited-memory history in TAO/LMVM

The two algorithms of Panels (a) and (b) were chosen to make the comparison clean, not to be representative. As we know, SciPy cannot be given a different inner product at all, and the Hilbert-space L-BFGS was written by us precisely so that it would use the right one. So, neither of these tells us what a user of this software stack would actually use.

Something that the user could meet is TAO. This is the optimisation component of PETSc, which already underpins every linear and nonlinear solve in Firedrake, so it comes with any Firedrake installation rather than being an extra dependency, and pyadjoint ships a wrapper that drives it directly. It also offers a whole library of methods rather than a single one. It is also one of the three packages Schwedes et al. [13, Sect. 2.1] name as hard-coded to the Euclidean inner product. It is therefore the natural place to ask whether that is still true.

The method we use throughout this subsection is TAO's `lmvm`, short for limited-memory variable metric, which is its own implementation of the limited-memory BFGS of Section 10.6. There is an obvious way to check whether it can be given the right inner product. Declaring the control as `Control(m, riesz_map="L2")` makes pyadjoint install the discrete L^2 Riesz map inside the solver, so if the inner product were the only obstacle, that single argument should be enough to reproduce the flat Moola row. It is not. Table 10 gives the counts on the deterministic graded meshes of Table 6, with every other setting left at its default.

target r	# cells	iterations
4	256	83
8	640	173
16	1 712	156
32	5 776	156
64	22 608	156
128	87 312	156

Table 10: TAO/LMVM on the Poisson control problem at PETSc's default limited-memory history of five, with the control declared as `Control(m, riesz_map="L2")` so that the mass-matrix seed is installed. Every run converged at $\varepsilon = 10^{-7}$.

Note that this is not what we expected. The counts are large, but they do not grow with the mesh. They rise once, from 83 to 173, and then settle at 156 even as the mesh ratio goes up by a further factor of eight and the cell count by a factor of fifty. The seed is doing its job, in that the run is already somewhat mesh-independent, but what is wrong is the size of the constant rather than its growth. So, we can see that the metric does indeed reach the algorithm, but there is still some fault, which we investigate.

It could be that the stall was a noise floor, a level below which round-off in the adjoint tape swamps the true gradient, so that no amount of further iteration helps. A noise floor belongs to the gradient computation rather than to the optimiser, so every solver reading the same tape would meet the same wall, and that is what makes it testable by changing the solver. We replaced TAO/LMVM with TAO/NLS, an exact Newton method that builds each step from conjugate-gradient (CG) Hessian-vector products, and tightened the tolerance to $\varepsilon = 10^{-30}$. The Poisson problem is quadratic, so a single Newton step lands on the discrete optimum. At an optimum the true gradient is zero, so whatever the tape reports there is nothing but numerical

error, which is exactly the quantity we want to measure. It reported $\|\mathcal{R}_{L^2}(J')\|_{L^2} \approx 10^{-18}$ on every mesh in the sweep. The floor is therefore at machine precision, far below where LMVM had stalled, and since both solvers read the same tape it cannot be what stopped LMVM. The same run says something more useful. An exact-Hessian method reaches the optimum in a single step, so the problem itself is not hard; what is expensive must be the way LMVM approximates that Hessian.

A second idea is that the limited memory is at fault, in that a rank-five quasi-Newton update is too coarse for the Hessian spectrum on a graded mesh. Our first test of this was itself invalid, because it used a synthetic exponential stretch, which collapses cells onto the Dirichlet corner where both the state u and the target d vanish. The over-resolved region there carries no information about the objective, so part of the apparent stalling came from the setup rather than from the solver.

The Newton probe points squarely at the limited memory. A rank-five update simply cannot capture enough of the Hessian spectrum on a graded mesh, so each step is poor and many of them are needed, but the metric is still being respected throughout. The fix is therefore to keep the seed and lengthen the history.

The history size is awkward to set in practice. Passing `tao_lmvm_mat_lmvm_hist_size` through pyadjoint's `parameters` dictionary does not propagate to the LMVM sub-matrix, because of an options-prefix mismatch between pyadjoint and the internal PETSc matrix object attached to the LMVM `Tao`. The option must instead be registered on the global PETSc options database before the `TAOSolver` is constructed,

```
PETSc.Options().setValue('-tao_lmvm_mat_lmvm_hist_size', 100)
```

or, alternatively, the pyadjoint TAO solver may be constructed with an empty options prefix. We use the former throughout.

Setting the history to 100 (and leaving everything else the same) gives Table 11.

target r	# cells	iterations
4	256	17
8	640	17
16	1 712	17
32	5 776	17
64	22 608	17
128	87 312	17

Table 11: The same runs as Table 10 with the limited-memory history raised from five to 100. Everything else, including the mass-matrix seed and the threshold, is unchanged.

The count is now flat at 17, and about nine times cheaper than before. Comparing the two tables, lengthening the history did not turn a mesh-dependent method into a mesh-independent one, because it was already mesh-independent. It made a mesh-independent method much cheaper in iteration count.

So TAO can be made to respect the function-space inner product after all, and the two tables separate the two ingredients that matter. The seed carries the metric and is what makes the count flat; the history decides how large that flat count is. There is a limit to what these two tables can show, however. Because `lmvm` is seeded in every run above, they vary the history alone, and so cannot tell us what the seed contributes by itself. For that we need a solver that has no seed, which is the experiment of Section 10.11.

10.10 A survey of the TAO solvers

Fixing `lmvm` raises a larger question. TAO is a library of optimisers rather than a single method, and a user picks one by name. If some honour the function-space inner product and others cannot, that is worth knowing before choosing. So, we run the same Poisson problem and grading sweep through every PETSc TAO solver that accepts an unconstrained, or bound-constrained, problem of this class, and collected the counts in Table 12.

Each row is a different optimiser that TAO provides, and the second column gives the short name used to select it. They fall into three families: the Newton-class solvers, which use the exact Hessian through Hessian-vector products, the quasi-Newton solvers, which build an approximation to it from successive gradients, and `cg`, a nonlinear conjugate gradient method, which keeps no Hessian information at all. A leading `b` marks a variant that also supports bound constraints.

type	TAO solver	$h_{\max}/h_{\min}$			
		4	16	64	128
Newton line search	<code>nls</code>	2	2	2	2
bounded Newton line search	<code>bnls</code>	1	1	1	1
bounded Newton trust region	<code>bntr</code>	8	8	9	9
bounded Newton trust + line	<code>bntl</code>	8	8	9	9
bounded Newton Krylov	<code>bnk</code>		n/a		
quasi-Newton (LMVM, history 100)	<code>lmvm</code>	17	17	17	17
bounded quasi-Newton + line	<code>bqnls</code>	30	76	130	163
orthant-wise quasi-Newton	<code>owlqn</code>		n/a		
nonlinear conjugate gradient	<code>cg</code>	1180 [†]	3758 [†]	> 20 000 [†]	n/a

Table 12: TAO solver iteration counts on the Poisson optimal control problem of Section 10.7, on the deterministic graded meshes of Table 6, at convergence threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{L^2}(J')\|_{L^2}$. The `lmvm` row uses the enlarged history of 100. The entries marked n/a , and the `cg` row marked [†], are explained below.

Three of the rows need a word of explanation. `bnk` requires explicit bound vectors, so it cannot run on this unbounded problem at all. `owlqn` is built for L^1 -regularised objectives, and it reports immediate convergence at $m = 0$ under its own L^1 optimality condition, which is not the optimum of the L^2 Tikhonov problem we are solving; neither solver returns a usable count. The `cg` row is the interesting one. On every mesh it stops at PETSc’s `DIVERGED_MAXFCN` limit after about 450 iterations, which is the cap on function evaluations rather than the 2000-iteration cap. With that budget raised it does converge on the coarser meshes. Table 13 shows what was measured.

$h_{\max}/h_{\min}$	# cells	iterations	factor
4	256	1 180	–
8	640	1 695	1.4
16	1 712	3 758	2.2
32	5 776	10 107	2.7
64	22 608	> 20 000*	–
128	87 312	<i>not attempted</i>	–

Table 13: Nonlinear conjugate gradient on the Poisson control problem of Section 10.7, at $\varepsilon = 10^{-7}$ with PETSc’s function-evaluation budget raised so that the runs are not cut short. The last column is the factor by which the count grows from the row above. *Stopped after more than nine hours without reaching the tolerance.

Two things justify stopping where we did. The first is cost. Each iteration needs a forward solve and an adjoint solve, so the $r = 32$ run took forty minutes, and the $r = 64$ run was still short of the tolerance after more than nine hours. Because the growth factor is itself increasing, from 1.4 to 2.2 to 2.7 across the sweep, a run at $r = 128$ on four times as many cells would take days rather than hours.

The second is that further runs would not change the conclusion. The four converged points already span nearly an order of magnitude, on meshes where every other solver in Table 12 finishes in under 130 iterations. Nor could the growth be removed by either of the means used in this dissertation, since `cg` builds no limited-memory seed to carry a metric and performs no inner Krylov solve for a preconditioner to act on. Measuring more precisely how badly it degrades would not make it any easier to fix.

The pattern across the table is clear, with one caveat. The Newton-class solvers are flat, and so is `lmvm` once its history is fixed, while nonlinear conjugate gradient grows steeply. The caveat concerns the Newton rows. What is counted there is outer Newton steps, and an exact Newton step does not depend on the inner product at all, so a flat count is what one would expect whatever metric is used. Each of those steps hides an inner Krylov solve that this table does not count, and Section 12 shows that the inner solve grows from 61 to 1052 iterations across this same sweep. For the Newton-class solvers the mesh-dependence has moved out of view rather than gone away. Whatever causes the growth is therefore not peculiar to LMVM, and it is not shared by every solver either. One row is left over: `bqnls` is mesh-dependent even though it is a quasi-Newton method like `lmvm`, and we will look into this further in Section 10.11.

One caveat applies to all of this. The Poisson problem is quadratic in m , and quasi-Newton methods do unusually well on quadratics whatever inner product they use, so the survey ought to be repeated on a problem that is not quadratic. The obvious candidate is the one-dimensional Burgers control problem, and we attempted it, but that run used the synthetic exponentially-stretched interval whose cells collapse onto the Dirichlet endpoint, the same problem as above, so its results are not included. A direct replacement is not available, because the graded-mesh construction used here is two-dimensional. Later work could cluster cells towards an interior point of the interval instead, or move to a two-dimensional Burgers problem on the same meshes as the Poisson problem.

On the Poisson evidence, then, what decides the outcome is the metric rather than the family a solver belongs to. Every solver here that can be given the function-space inner product is flat, and every one that cannot is not. The Newton-class rows only appear to escape this, and the price they pay is deferred to the inner solve that Section 12 takes up.

10.11 Isolation of the seed in `bqnls`

The diagnosis above leaves a question that deserves its own treatment, namely why installing the mass-matrix seed makes `bqnls` mesh-independent. This section and the next answer it, using the conditioning machinery that Schwedes et al. [13, Sect. 2.2] developed for steepest descent. We first pin down experimentally which ingredient does the work.

Two pieces of notation are needed for what follows. TAO calls the initial Hessian approximation of a limited-memory method its J_0 , and this is what we have been calling the seed. Setting $J_0 = M_h$, with M_h the finite element mass matrix of Section 9.1, therefore starts the recursion from the mass matrix rather than from the identity, and it is the mass matrix because M_h^{-1} is the discrete L^2 Riesz map.

The two candidate ingredients are the seed and the history length, so we ran `bqnls` in all four combinations on the deterministic graded meshes of Table 6, at the usual threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{L^2}(J')\|_{L^2}$. The results are collected in Table 14.

seed	history	$h_{\max}/h_{\min}$			
		4	16	64	128
none	5 (default)	30	76	130	163
none	100	23	49	80	–
$J_0 = M_h$	5	6	6	6	6
$J_0 = M_h$	100	6	6	6	6

Table 14: Iteration counts for `bqnls` on the Poisson optimal control problem of Section 10.7, in the four combinations of seed and history. The seed is necessary and sufficient for mesh-independence, and the history is neither. Without the seed the count grows with the mesh ratio at either history, and with the seed it is a flat six at either history. Every seeded run converged on the gradient tolerance. The missing entry was not run.

The table separates the two ingredients completely. Without the seed the count grows with the mesh ratio whether the history is five or one hundred, so no amount of memory rescues the run. With the seed the count is a flat six whether the history is five or one hundred, so almost no memory is needed. The seed does all of the work, and whatever the seed provides, six directions’ worth of memory is enough to exploit it.

What the seed provides is an inner product. The two-loop recursion pairs a primal step with a dual gradient difference, which is a duality pairing and carries no metric. The single metric-carrying operation is the seed, applied between the two sweeps, as we will confirm by reading the source in Section 11.3. Installing $J_0 = M_h$ makes the recursion apply $J_0^{-1} = M_h^{-1} = \mathcal{R}_{L^2}$, the discrete Riesz map, at exactly that point. By Schwedes et al. [13, (1.208)–(1.210)] this turns the Euclidean L-BFGS update into the Hilbert-space update in L^2 . Seeded `bqnls` is therefore not a better-tuned ℓ^2 method but one applying a different operator, the same update as the hand-written reference of Section 10.6, though the two still differ in their line search and in the bound handling. Consistently with that, it took the same six iterations as that reference did in Table 8, which was itself run with a history of five.

One distinction is worth making clear. This is the same operator M_h^{-1} that Section 12.4 installs as a preconditioner on the inner Krylov solve of the Newton solver, but it enters at a different point. Since `bqnls` performs no Hessian solve, there is no Krylov iteration to precondition, and the metric can only be supplied as the quasi-Newton seed. The same Riesz map is supplied at two different points of two different algorithms; the principle connecting them is that choosing an inner product and preconditioning are the same act [13, pp. 55–56], which is the classical

operator-preconditioning principle [6, 8].

10.12 The reduced Hessian in the two inner products

The previous section showed by experiment that the seed is what makes the count flat, and that no amount of memory substitutes for it. This section explains why, by looking at the spectrum of the Hessian that each run actually sees. The argument is the one Schwedes et al. [13, Sect. 2.2] give for steepest descent, applied here to a quasi-Newton method, and it also accounts for the more surprising half of the experiment, namely why a longer history cannot make up for a missing seed.

The Poisson problem is quadratic in the control, and the behaviour of a BFGS-type method on a quadratic is governed by the spectrum of the Hessian measured in the inner product the method works in. The two halves of Table 14 therefore see two different spectra of the same problem. Write K_h and M_h for the stiffness and mass matrices, and $\mathbf{m}$ for the coefficient vector of the control. The discrete state is then $U = K_h^{-1} M_h \mathbf{m}$, and the Hessian of the discrete reduced functional, as a function of $\mathbf{m}$, is

$$H = M_h K_h^{-1} M_h K_h^{-1} M_h + \alpha M_h = M_h (S_h^2 + \alpha I), \quad S_h := K_h^{-1} M_h. \quad (116)$$

This factorisation is the key point, since every cell-size scaling in H sits in the leading mass-matrix factor, exactly (not just approximately) so for the regularisation term αM_h , while the remaining factor $S_h^2 + \alpha I$ is the matrix of the reduced Hessian operator in the L^2 inner product.

The unseeded run works in ℓ^2 , so it sees the spectrum of H itself, which inherits the conditioning of M_h . This is precisely the mechanism that Schwedes et al. [13, Sect. 2.2.4] analyse for steepest descent. For their model problem the Hessian *equals* the mass matrix, and Schwedes et al. [13, Remark 2.2.4.1] extends the analysis to any objective containing an $\alpha(u, u)_H$ term, as ours does. The local mass matrix of an element scales with its volume, $M_K = h_1 \cdots h_n \hat{M}$ [13, (2.36)], where $\hat{M}$ is the reference-element mass matrix. This gives the condition number bound

$$\kappa(M_h) \leq p_{\max} \frac{\lambda_{\max}^{\hat{M}}}{\lambda_{\min}^{\hat{M}}} \left(\frac{h_{\max}}{h_{\min}} \right)^n, \quad (117)$$

with $p_{\max}$ the maximum number of elements meeting at a node [13, (2.40)], and from it an iteration count for steepest descent that is polynomial in $h_{\max}/h_{\min}$ of order the domain dimension n [13, (2.44)]. On our two-dimensional meshes the bound grows like $(h_{\max}/h_{\min})^2$, reaching roughly 1.6×10^4 at $r = 128$. Schwedes et al. [13] derive the estimate for steepest descent rather than for L-BFGS, so for **bqnls** we take from it the mechanism rather than the exponent; the observed unseeded row (30, 76, 130, 163) grows with the ratio exactly as the mechanism predicts.

The same picture shows why a longer history cannot substitute for the seed. A limited-memory method with memory ℓ corrects its seed by an update of rank at most 2ℓ , so it can repair a spectrum that is wrong in a few directions. The spread of M_h is not of that kind, since the cell volumes vary continuously across the graded mesh, one scale per region and thousands of cells in all, so no moderate-rank correction of the identity can approximate M_h^{-1} . The history-100 row of Table 14 does keep growing.

The seeded run works in L^2 , so it sees the generalised spectrum of H relative to M_h , which by equation (116) is the spectrum of $S_h^2 + \alpha I$. The matrix S_h discretises the solution operator $S = (-\Delta)^{-1}$, which is compact. The spectrum is therefore a cluster at α together with a small number of decaying outliers, and it converges to the spectrum of the continuous operator as the mesh refines, however the mesh is graded. This is the same clustered spectrum that makes the

Riesz-preconditioned inner CG mesh-independent (Section 12.3); here it acts one level up, on the outer quasi-Newton iteration. The seeded run therefore faces a fixed handful of significant directions, whatever the grading, which is why its count is small, flat, and already attainable with a memory of five.

One detail remains open. The `lmvm` solver, which carries the same seed, is also flat at history five but at about 156 iterations (Table 10), and needs a history of 100 to come down to 17 (Table 11), where seeded `bqnls` manages six with a history of five. We suspect the scalar seed rescaling, which TAO disables for `lmvm` when J_0 is a matrix (Section 11.3), but we have not confirmed this.

11 Source-Level Comparison of the L-BFGS Implementations

Section 10 established by experiment that installing the mass-matrix seed changes how TAO/LMVM behaves. An iteration count, though, tells us only what came out, and not what the algorithm computed to get there. This section sets the limited-memory BFGS of PETSc TAO beside the two formulations of Schwedes et al. [13, Sect. 1.5.4], the Euclidean and the Hilbert-space, and beside the Moola reference implementation, at the level of the actual recursion, asking where an inner product enters each of them and whether the four are really doing the same thing.

There are two reasons to look this closely. The first is that the failure of Section 10.9 looked for a long time like a defect inside TAO, and an accusation of that kind should not be left resting on iteration counts. The second is that if a metric is to be handed to an optimiser at all, we need to know where in the algorithm an inner product is used, and that is a question we will want to put to a different solver in Section 12. The source was read against PETSc v3.24.0 and Moola’s reference L-BFGS implementation;⁵ the structural conclusions are version-independent, though the specific line references are not.

11.1 The common skeleton

All four algorithms compute the same object, the action of an approximate inverse Hessian on the gradient, written $H_k g$. None of them ever builds H_k as a matrix, because only its action on a vector is ever needed [13, Sect. 1.5.4.2]. They all compute that action the same way, by a two-loop recursion that sweeps backwards through the stored history and then forwards again.

Between the two sweeps sits a single operation, the initial inverse Hessian H_0 , which we call the *seed*. Every other operation in the recursion pairs a gradient with a step, and a pairing of that kind needs no inner product to make sense. Written so that the seed stands on its own, the shared recursion is

```
function direction(g):          # returns -H_k g
    q = g
    for j = newest ... oldest:  # first sweep
        a[j] = rho[j] * <s[j], q>    # pairing: step . q
        q = q - a[j] * y[j]
    z = H0(q)                  # SEED: the one metric step
    for j = oldest ... newest:    # second sweep
        b = rho[j] * <y[j], z>      # pairing: grad-diff . z
        z = z + (a[j] - b) * s[j]
    return -z
```

⁵Specifically `src/ksp/ksp/Utils/lmvm/` in PETSc and `moola/algorithms/bfgs.py` in Moola.

The four implementations differ only in how they fill three slots, namely the pairing, the curvature scalar ρ , and the seed.

11.2 The two Schwedes formulations

Schwedes et al. [13] give the recursion twice. In the Euclidean form [13, (1.187), (1.209)] the control is a coefficient vector, every pairing is the ordinary dot product, and the seed is a multiple of the identity, $H_0 = \gamma_k I$ with $\gamma_k = (s_{k-1}^\top y_{k-1}) / (y_{k-1}^\top y_{k-1})$. Note that the denominator pairs two derivative objects with nothing between them.

In the Hilbert-space form [13, (1.205), (1.210)] the control lives in M while the derivative lives in the dual space M^* . These are different spaces, so a pairing can no longer be a dot product. It is instead a duality pairing, a derivative evaluated on a step, written $y_k(s_k)$. The seed has more work to do as well, since it must carry an object of M^* back into M , and once an inner product has been chosen the operator that does this is the Riesz map.

The two forms differ in the seed and nowhere else, and Schwedes et al. [13, (1.208) and following] says so outright. Taking $M = \mathbb{R}^n$ with the dot product makes the Riesz map the identity, and the Hilbert form collapses onto the Euclidean one. For $L^2(\Omega)$ or $H^1(\Omega)$ controls it does not.

The hand-written reference of Section 10.6 writes the Hilbert algorithm in its *primal-representer* form. It pre-applies $\mathcal{R}_H$ to the derivative, so that $g_k := \mathcal{R}_H(d\tilde{J}(m_k))$ is primal, and takes all pairings in $(\cdot, \cdot)_H$. This is identical to the dual, pairing form above, since pre-applying $\mathcal{R}_H$ and using $(\cdot, \cdot)_H$ is the same as keeping the derivative dual and using $\mathcal{R}_H$ only as the seed; the two are the two sides of the same coin of the Riesz representation theorem (Section 2.4)⁶.

11.3 The TAO/LMVM recursion

TAO's limited-memory driver is an ordinary quasi-Newton loop. It computes the gradient, builds a direction from the stored history, checks that the direction points downhill, runs a line search, and repeats. The history stores the step $s = X - X_{\text{prev}}$ and the gradient difference $y = F - F_{\text{prev}}$, keeping a pair only when $s^\top y > 0$.

Two features of the implementation obscure the recursion without changing what it computes. The BFGS direction is produced by the DFP kernel run in dual mode, which exploits a duality between those two updates, so the roles of s and y look reordered against a textbook two-loop even though the operator is the same. The sweeps are also batched, so that a loop of dot products becomes a single matrix-vector call.

What matters for us is what those dot products pair together. Each of them pairs the dual y , or the gradient, with the primal s , or the running vector. That is a duality pairing, and in coefficients a duality pairing is just the ordinary dot product. The bare dot is therefore the correct operation, and it carries no metric of its own.

The seed is applied as a linear solve against a matrix TAO calls J_0 . If no J_0 is supplied the seed is the scalar $\gamma_k I$. If $J_0 = M_h$, which is what `Control(m, riesz_map="L2")` installs, the solve computes $M_h^{-1}q$, so the seed is exactly the discrete L^2 Riesz map. TAO also switches off its scalar rescaling whenever J_0 is a full matrix rather than a diagonal one, so the Riesz seed is applied unscaled.

⁶Two of the printed equations appear to contain slips. In (1.205) the final rank-one term is printed without its denominator $y_k(s_k)$, which the primal and dual types require and which Moola's code carries; in (1.210) the last denominator is printed with s_{k-2} where consistency with (1.209) calls for $y_{k-1}(s_{k-1})$.

11.4 The Moola recursion

Moola keeps primal and dual vectors as separate types, so its code makes the distinction that the mathematics requires. Applying a dual object to a primal one is the duality pairing, and converting a dual object to a primal one is the Riesz map. Its recursion is the recursive form of Schwedes et al. [13, (1.210)], with the curvature scalar $\rho = 1/y(s)$ written out explicitly, which independently confirms the denominator discussed above.

Two of its defaults are worth noting, because they are the opposite of TAO's. Moola's seed is the Riesz map already, rather than a scaled identity, and it scales that seed using the dual norm rather than the Euclidean one. It also tests convergence in the dual norm by default.

11.5 The four implementations in pseudo-code

Here are the algorithms for the four implementations, written as the skeleton of Section 11.1 with each one's choices filled in.

```
# --- Schwedes Euclidean (Eq 1.187 / 1.209) ---
function direction_schwedes_Rn(g):
    q = g                                     # g, s, y all in R^n
    for j: a[j] = (s[j].T @ q) / (y[j].T @ s[j]) # Euclidean dot
        q -= a[j] * y[j]
    gamma_k = (s[k-1].T @ y[k-1]) / (y[k-1].T @ y[k-1])
    z = gamma_k * q                          # SEED: gamma_k * I
    for j: b = (y[j].T @ z) / (y[j].T @ s[j])
        z += (a[j] - b) * s[j]
    return -z
```

```
# --- Schwedes Hilbert (Eq 1.205 / 1.210) ---
function direction_schwedes_H(g):
    q = g                                     # g, y in M^*; s in M
    for j: a[j] = q(s[j]) / y[j](s[j])       # duality pairing
        q -= a[j] * y[j]
    gamma_k = y[k-1](s[k-1]) / y[k-1](R_H(y[k-1])) # dual norm
    z = gamma_k * R_H(q)                     # SEED: scaled Riesz
    for j: b = y[j](z) / y[j](s[j])
        z += (a[j] - b) * s[j]
    return -z
```

```
# --- Moola (recursive form of Eq 1.210) ---
function direction_moola(g):
    q = g                                     # dual
    for j: a[j] = s[j].apply(q) / y[j].apply(s[j])
        q.axpy(-a[j], y[j])
    theta = y[k-1].apply(s[k-1]) / y[k-1].apply(y[k-1].primal())
    z = theta * q.primal()                   # SEED: theta * Riesz
    for j: b = y[j].apply(z) / y[j].apply(s[j])
        z.axpy(a[j] - b, s[j])
    return -z
```

```
# --- TAO LMVM (with Control(m, riesz_map="L2")) ---
function direction_tao_lmvm(g):
    q = g                                     # PETSc Vec of coefficients
    for j: a[j] = VecDot(s[j], q) * rho[j] # ell^2 = duality pairing
        VecAXPY(q, -a[j], y[j])
    # no scaling term: TAO disables it when J_0 is a matrix
    z = M_h^{-1} q                          # SEED: KSPSolve(J_0=M_h)
    for j: b = VecDot(y[j], z) * rho[j]
        VecAXPY(z, a[j] - b, s[j])
    return -z
```

Reading down the column of seed lines, only those lines differ; the sweeps are structurally identical. The Schwedes-Euclidean and Schwedes-Hilbert variants differ only in whether the seed is $\gamma_k I$ or $\gamma_k \mathcal{R}_H$, consistent with Schwedes et al. [13, (1.208)] reducing one to the other when $\mathcal{R}$ is the identity. Moola and TAO-with- L^2 differ only in the scalar θ against an unscaled seed. Moola scales the Riesz seed by a dual-norm-correct θ , while TAO uses the bare Riesz seed because its scalar rescaling is auto-disabled when J_0 is a matrix (Section 11.3).

The differences are summarised in Table 15.

Item	Schwedes $\mathbb{R}^n$	Schwedes Hilbert	Moola	TAO (v3.24)
step s_k	$x_{k+1} - x_k$	$m_{k+1} - m_k \in M$	αp_k (primal)	$X - X_{\text{prev}}$
grad diff y_k	$\nabla f_{k+1} - \nabla f_k$	$d\tilde{J}_{k+1} - d\tilde{J}_k \in M^*$	$g_{k+1} - g_k$ (dual)	$F - F_{\text{prev}}$
curvature	$y_k^\top s_k$	$y_k(s_k)$ pairing	<code>y.apply(s)</code>	<code>VecMDot</code>
sweep pairings	Euclidean dot	duality pairing	<code>.apply()</code>	<code>VecDot</code>
recursion	two-loop	two-loop ($\mathcal{V}_k, \mathcal{V}_k^*$)	recursive	batched, DFP-dual
seed, default	$\gamma_k I$	$\gamma_k \mathcal{R}_H$	$\mathcal{R}$ (Riesz)	$\gamma_k I$
seed, Riesz set	n/a	$\mathcal{R}_H$	$\mathcal{R}$	M_h^{-1}
seed scaling	via $y^\top y$	via $(y, y)_{M^*}$	dual-norm	off if $J_0 = M_h$
conv. norm	ℓ^2	dual	dual	ℓ^2 default

Table 15: The four limited-memory BFGS implementations differ only in how the single inner-product choice is realised. Every row above the seed is a metric-free pairing; the algorithms diverge at the seed H_0 and in the (convergence-only) norm. The Euclidean formulation has no Riesz map to set, since its control space is $\mathbb{R}^n$ under the dot product, so that entry is marked n/a .

11.6 Correctness of the TAO/LMVM recursion

The recursion is structurally correct. Nowhere does it pair two dual objects together with the Riesz map missing between them, which is the error one would look for. Every pairing puts a dual object with a primal one, and the metric enters at the seed and nowhere else. With $J_0 = M_h$ the seed is the L^2 Riesz map, and TAO applies the same update as Schwedes et al. [13, (1.210)], as Moola and as the reference of Section 10.6, up to the seed scaling noted below. The experiments of Section 10.9 support this. There, `lmvm` carries the mass-matrix seed in every run and is mesh-independent at either history, flat at about 156 iterations by default and at 17 once the history is enlarged (Tables 10 and 11). The limited-memory truncation therefore governs the cost, and the seed governs the mesh-independence, neither of which points to an inner-product bug. The related `bqnls` solver reinforces the role of the seed itself, since `pyadjoint` installs none for it, and it stays mesh-dependent until the seed $J_0 = M_h$ is added by hand (Section 10.11), which directly shows the point of this section, that the metric enters at H_0 .

Even so, the correctness depends on assumptions the recursion makes rather than checks. The key one is that the gradient reaches TAO as a dual object and s as a primal one. The argument that `VecDot` is the right pairing fails if `pyadjoint` pre-applies the Riesz map and hands TAO a primal gradient, as the reference of Section 10.6 deliberately does, for then the same dots become primal-with-primal and *are* missing an inverse. This is a fact about the `pyadjoint` interface rather than the TAO source, and is the one premise that could invert the verdict, so it is worth confirming directly. The remaining premises are milder. The Riesz solve must be exact. Here, M_h^{-1} is applied by a `KSPSolve` on J_0 , which a direct LU makes exact, whereas an iterative solve with a weak preconditioner applies a different operator and the metric is no longer clean (the same caution as for the Riesz solve in Section 10.6). The space must be right, since $J_0 = M_h$ gives the L^2 Riesz map only, while an H^1 control needs $J_0 = M_h + K_h$, a Helmholtz solve. And the variant must be right, since the analysis is for `MATLMVMBFGS` in the `recursive`

multiplication mode, whereas the `compact_dense` and `dense` variants form Gram-type products $Y^T Y$ that reintroduce cofunction-cofunction dots and would need separate checking.

Even when the algorithms are matched, residual differences remain, though these affect the iteration count rather than correctness. Moola scales its Riesz seed by a dual-norm-correct `theta` while TAO uses M_h^{-1} unscaled; Moola tests convergence in the dual norm by default while TAO uses ℓ^2 unless `TaoSetGradientNorm` is set, which changes *when* the solver stops rather than *its* iterates; and, although both use a Moré–Thuente strong-Wolfe line search, so this is not itself a source of difference, their default history lengths differ, five against ten. The practical failure was therefore not a correctness bug. The original inability to reproduce the Moola row (Section 10.9) was the limited-memory history being too short for the discrete-Hessian spectrum on graded meshes, not an internal ℓ^2 inner product.

Two structural points follow from this reading of the source, and both matter for Section 12. The first is that it is `setLMVMHO` rather than `setGradientNorm` that decides the outcome, since the seed determines the search direction and with it the mesh-dependence, whereas the gradient norm only weights the convergence test. The second is that the recursion is matrix-free apart from the seed, never assembling H_k , so the one genuine linear solve it performs is M_h^{-1} at the seed itself. A limited-memory method therefore has no inner Krylov solve to precondition, which is why the metric has to be supplied at H_0 , whereas the Newton method of Section 12 does have one, and there the metric enters as a preconditioner instead.

12 Two-Level Riesz-Map Preconditioning

The experiments of Section 10 reproduced the finding of Schwedes et al. [13] that a quasi-Newton optimiser becomes mesh-independent once its inner product is the Riesz map of the control space. In a limited-memory method that inner product enters as the initial-Hessian seed. All of this, though, concerns the outer optimiser alone.

This section takes up what happens one level down. A Newton method computes each search direction by solving a linear system, and that solve is itself iterative. We show that the same mesh-dependence reappears there, in the inner Krylov solve, and that the same idea removes it, namely preconditioning that solve with the Riesz map.

That an inner product can act as a preconditioner is not itself new. The idea is already present in classical operator preconditioning [6, 8], and it was raised in discussion among PETSc/TAO developers.⁷ What has not been done before, to our knowledge, is to expose and remove the effect experimentally in an adjoint optimisation pipeline built on pyadjoint.

The conclusion is that the metric has to be supplied twice, as the seed on the outer iteration and as the preconditioner on the inner solve, and that mesh-independence needs both. We first expose the inner-level growth and remove it with an L^2 Riesz-map preconditioner, and we have prepared a production version of it for pyadjoint. We then show that this preconditioner must be built from the Riesz map matching the regularisation, and check that the growth left by a mismatched map is genuine conditioning rather than a side-effect of how the runs were stopped.

12.1 A mesh-dependence hidden inside TAO/NLS

The `nls` row of Table 12 shows two outer iterations on every mesh in the sweep and so appears mesh-independent, but actually this hides a mesh-dependence one level deeper. At each outer Newton step, `nls` solves $H p_k = -g_k$ for the search direction, where H is the discrete reduced

⁷PETSc/TAO developers, personal communication, 2025 [7]; in particular the observations that, for linear solvers, “changing the inner product” is exactly preconditioning, and that an inexact inner solve needs a mesh-independent preconditioner together with a preconditioned norm.

Hessian and g_k the assembled dual gradient. That system is not solved directly but by an inner Krylov iteration, the conjugate gradient method. Written in the underlying ℓ^2 coefficient inner product, H is badly conditioned on graded meshes for the same reason the outer L-BFGS Hessian is (Section 10.7): the cell-size scaling carried in the diagonal of the mass matrix. The inner CG iteration count per outer Newton step therefore grows with the grading, even though the outer Newton count does not. This is the problem identified by Schwedes et al. [13], repeated one level down.

To measure the hidden growth, we ran `nls` on the Poisson optimal control problem of Section 10.7, on four of the six deterministic graded meshes of Table 6. The inner KSP was left unpreconditioned (`-tao_nls.ksp_pc_type none`), and a counting monitor on it summed the CG iterations across both outer Newton steps. The totals are given in Table 16.

$h_{\max}/h_{\min}$	# cells	inner CG iterations
4	256	61
16	1 712	179
64	22 608	573
128	87 312	1052

Table 16: Total inner-CG iterations for TAO’s `nls` on the Poisson control problem of Section 10.7, with no preconditioner on the inner solve. The outer Newton count is two on every mesh, so all of this variation is hidden inside those two steps.

They grow from 61 at $r = 4$ to 1052 at $r = 128$, roughly doubling at each refinement, which is the growth that the flat `nls` row of Table 12 conceals.

12.2 Preconditioning of Krylov methods

The growth in Table 16 is a conditioning problem, and the usual way to deal with poor conditioning in a Krylov solver is preconditioning. We define it here before choosing one. An iterative linear solver such as conjugate gradients does not invert the system matrix A in $Ax = b$; it takes a sequence of cheap steps that progressively drive the residual to zero. The number of steps it needs is governed by the *conditioning* of A , that is, loosely by how widely its eigenvalues are spread. When the eigenvalues are tightly clustered the solver converges in a handful of steps; when they span many orders of magnitude it takes many. Preconditioning replaces the system by an equivalent one, $P^{-1}Ax = P^{-1}b$, with the same solution but a better-conditioned operator, so the solver converges far faster. The preconditioner P must be cheap to apply, and should approximate A well enough that $P^{-1}A$ has clustered eigenvalues. Taking $P = A$ would converge in a single step but is as costly as the original problem. The trick is to capture the source of the poor conditioning while staying cheap. In our case the source of the poor conditioning is known exactly, which is what makes it possible to choose P by argument rather than by experiment.

12.3 Choosing the preconditioner

We now have a problem and a general way to attack it, but not yet a choice of preconditioner. That choice does not have to be made by trial and error. The spectrum of the operator which the inner CG solves says both where the growth of Table 16 comes from and what will remove it. The argument is the one Section 10.12 applied to the outer quasi-Newton iteration, now one level down. In the L^2 inner product the reduced Hessian is $\mathcal{H} = S^*S + \alpha I$, where S is the control-to-state solution map. Because S solves a Poisson problem it is smoothing, so S^*S is compact and its eigenvalues decay to zero. The operator $\mathcal{H}$ is therefore the identity scaled by α plus a compact perturbation, and its eigenvalues cluster at α with only finitely many outliers.

Its conditioning is bounded independently of the mesh, and that is exactly the situation in which conjugate gradients converges in a fixed number of steps.

What the inner CG sees without a preconditioner, though, is not $\mathcal{H}$ but its matrix in the ℓ^2 coefficient inner product, which carries an extra factor of the mass matrix M_h . On a graded mesh the diagonal of M_h ranges over the cell areas, so M_h is badly conditioned, and this scaling spreads the spectrum out and makes the CG count grow. Preconditioning by M_h^{-1} , the discrete L^2 Riesz map, removes exactly this factor and returns the CG to the mesh-independent spectrum of $\mathcal{H}$ itself. The prediction is therefore a sharp one: under that preconditioner the inner-CG count should stop growing with the mesh altogether. This is the finite-element example of a general principle, namely that the natural preconditioner for a discretised operator is the discrete Riesz map of the space on which it acts [6, 8].

The same picture predicts what should happen if the wrong map is chosen. Preconditioning the L^2 -natural operator by the H^1 Riesz map $(M_h + K_h)^{-1}$ does not cancel the mass matrix. It over-smooths instead, replacing the clustered spectrum of $\mathcal{H}$ by one whose smallest eigenvalues shrink as the mesh resolves higher frequencies, so the count grows again. Mesh-independence returns only when the regularisation is itself an H^1 norm, because the H^1 Riesz map is then the matching one. Section 12.6 puts both halves of this prediction to the test.

12.4 Preconditioning the inner CG of TAO/NLS

We now put that choice into practice. Preconditioning the inner CG with the discrete Riesz map $\mathcal{R}_{L^2} = M_h^{-1}$ cancels the cell-size scaling that ruined the conditioning, exactly as it does for the outer optimiser in Section 10.7. We first wrote this the textbook Firedrake way, as an `AuxiliaryOperatorPC` subclass returning the mass form, which should be selectable through a single options-database entry:

```
'tao_nls_ksp_pc_type': 'python',
'tao_nls_ksp_pc_python_type':
    'meshdep.preconditioners.RieszMapL2',
```

This silently failed, however. The `AuxiliaryOperatorPC` needs the operator matrix to carry a Firedrake function space so that it can build the mass form on the correct space. Pyadjoint's reduced-Hessian matrix is a Python `Mat` without one, so the preconditioner was left at type `none` with no warning. The workaround was to write a plain PETSc Python preconditioner instead, `RieszMapPCContext`, which takes the function space at construction, assembles M_h from it and factorises it by LU. It is wired directly onto the inner KSP after the `TAOSolver` is built:

```
solver = TAOSolver(MinimizationProblem(Jhat), parameters=...)
pc = solver.tao.getKSP().getPC()
pc.setType('python')
pc.setPythonContext(RieszMapPCContext(V, riesz_map="L2"))
solver.solve()
```

The diagnostic `tao_nls_ksp_view` then reported PC type: `python` with the Riesz context attached, confirming that the preconditioner was installed.

The baseline and Riesz- L^2 runs on the same four meshes are compared in Table 17 and plotted in Figure 12. Outer Newton stayed at two iterations in every row, since the Poisson problem is quadratic, so the entire row-to-row variation lies in the inner CG. The baseline totals grew $61 \rightarrow 179 \rightarrow 573 \rightarrow 1052$ across $r \in \{4, 16, 64, 128\}$, matching the shape of the SciPy ℓ^2 row of Table 8. The Riesz- L^2 totals were flat at 12 CG iterations on every mesh in the sweep, a reduction of roughly $88\times$ at $r = 128$.

target r	realised $h_{\max}/h_{\min}$	cells	baseline inner CG	Riesz L^2 inner CG
4	4.00	256	61	12
16	16.00	1 712	179	12
64	64.00	22 608	573	12
128	128.00	87 312	1052	12

Table 17: Total inner-CG iteration counts across both outer Newton steps for TAO’s `nls` on the Poisson optimal control problem, on the deterministic graded meshes of Table 6, at convergence threshold $\varepsilon = 10^{-7}$ on $\|\mathcal{R}_{L^2}(J')\|_{L^2}$. Outer Newton took two iterations in every row. The baseline column uses no preconditioner on the inner KSP; the Riesz- L^2 column uses a plain PETSc Python preconditioner applying M_h^{-1} by a direct LU factorisation of the mass matrix on the control space.

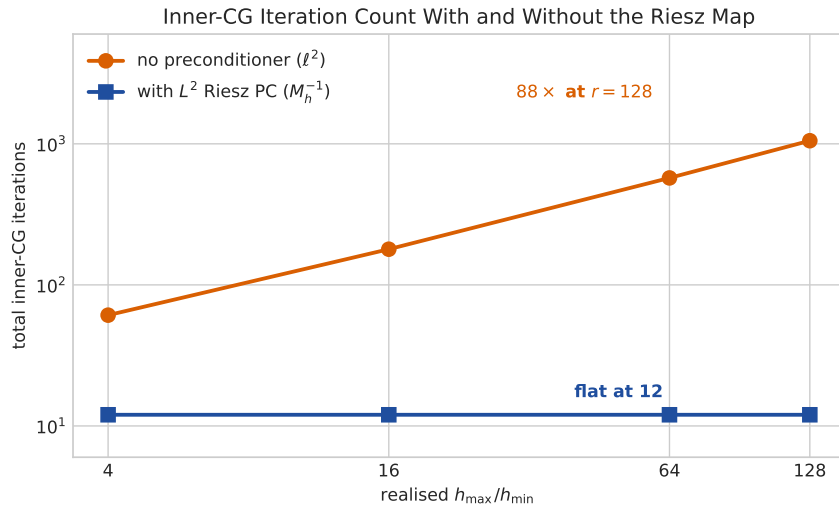


Figure 12: Total inner-CG iterations for TAO’s `nls` on the Poisson optimal control problem against the realised mesh ratio, on the deterministic graded meshes of Table 6 (the data of Table 17). Without a preconditioner the count grows polynomially with the mesh ratio; with the L^2 Riesz-map preconditioner it stays flat at twelve.

Table 17 is the direct counterpart of Table 8 at the level of the inner Krylov solve rather than the outer optimiser. The same failure, the bad ℓ^2 conditioning of a discrete operator on graded meshes, is removed by the same Riesz map, applied here as a Krylov preconditioner rather than as a quasi-Newton initial-Hessian seed. It also closes the loose end in the `nls` row of Table 12, whose flat outer counts were only half of the story.

12.5 A production-ready preconditioner for pyadjoint

The `RieszMapPCContext` used above takes the control function space as an explicit argument. That is fine for a single experiment, but awkward as a reusable component. The main software contribution of this work is a general version, the class `RieszMapPC`. It is a PETSc Python preconditioner attached to the reduced-Hessian Krylov solve inside TAO’s Newton method. Each time that solve calls its preconditioner it hands over a dual object, a residual in V^* , and `RieszMapPC` hands back the primal representative in V . That is the Riesz map $V^* \rightarrow V$. For the L^2 metric it means solving with M_h , and for the H^1 metric with $M_h + K_h$. That is exactly the operation which produced the flat count of Table 17, now performed by the library rather than by hand. What makes it reusable is where it finds that map: it reads the Riesz map directly from each `Control` and so needs no such argument. Because it applies whichever Riesz map a control

carries, the one preconditioner serves both the L^2 and the H^1 problem without modification. It is written for pyadjoint’s TAO solver interface, through which Firedrake drives optimisation with PETSc TAO. It also includes a fallback, so that it can still be imported where `petscctools` is absent and fails only if it is actually used. It is selected through the two options `tao_nls_pc_type = python` and `tao_nls_pc_python_type = pyadjoint.optimization.RieszMapPC`.

The preconditioner is covered by tests in two places. Our own checks install it on TAO’s inner Krylov solve and verify three things. First, that it is actually attached, with the diagnostic reporting `PC type: python` and the Riesz context present. Second, that it drives the inner-CG count below the unpreconditioned baseline. Third, that it reproduces the flat count of the hand-wired `RieszMapPCContext` exactly: on the Poisson problem of this section it matches Table 17 to the iteration. The regression test contributed to Firedrake is deliberately smaller. It selects the preconditioner through the options database on a one-dimensional model problem, asserts that it really is installed on the inner solve rather than silently ignored, and checks that the optimiser recovers the reference control. The preconditioner has been merged into pyadjoint, and the test into Firedrake. The two-level fix is therefore available to any pyadjoint user through the metric already attached to their `Control`, rather than through the manual plumbing of Section 12.4.

12.6 Matching the Riesz map to the regularisation

The inner-CG preconditioning of Section 12.4 used the L^2 Riesz map, which is the correct choice for the L^2 -regularised problem. To see that the choice must *match* the regularisation, we ran the same TAO/NLS solve on the interior random-refined meshes of Section 10.4, preconditioning the inner CG in turn with the L^2 map M_h^{-1} and the H^1 map $(M_h + K_h)^{-1}$. The change of mesh family is deliberate. Those meshes place their refinement at random rather than by a fixed pattern, so a result that survives on them cannot come from the particular way the graded meshes are built. They also reach the same realised ratios, which keeps the two experiments comparable. The L^2 -regularised results are given in Table 18, and Figure 13 plots them beside the mirror experiment described below.

level	realised $h_{\max}/h_{\min}$	cells	baseline	L^2 PC	H^1 PC
4	4.00	314	65	14	131
6	8.00	772	108	13	205
8	16.00	1 970	341	13	323
10	32.00	5 450	614	14	417
12	64.00	16 020	1105	13	522
14	128.00	46 998	2115	14	526

Table 18: Total inner-CG iteration counts for TAO’s `nls` on the L^2 -regularised Poisson optimal control problem, over six interior random-refined meshes, which reach the same realised $h_{\max}/h_{\min}$ as the graded meshes of Table 6, with no preconditioner (baseline) and with the L^2 and H^1 Riesz maps as inner-KSP preconditioners. Outer Newton took two iterations in every case. The L^2 map is flat; the H^1 map, mismatched to the L^2 regularisation, grows almost as fast as the baseline.

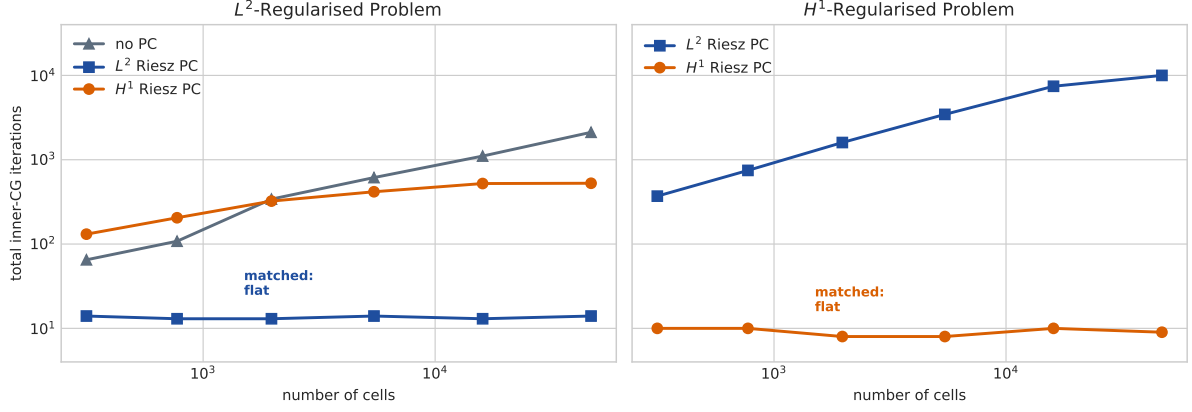


Figure 13: Total inner-CG iterations for TAO’s `nls` against the number of mesh cells, on the interior random-refined meshes, under the L^2 and H^1 Riesz-map preconditioners. *Left*: the L^2 -regularised problem (the data of Table 18). *Right*: the H^1 -regularised problem. In each case the map that matches the regularisation stays flat and the mismatched map grows, so the same two colours swap roles when the norm changes.

The L^2 map keeps the inner-CG count flat, reproducing on random meshes the behaviour of Table 17. The H^1 map, mismatched to the L^2 regularisation, grows with the mesh from 131 to 526, almost as fast as the unpreconditioned baseline.

Two checks confirm that this growth is genuine conditioning rather than a side-effect of stopping the runs early. First, the L^2 - and H^1 -preconditioned solves reach the *same* minimiser. At the third refinement level the two objective values agree to fifteen digits ($|J_{L^2}^* - J_{H^1}^*| = 3 \times 10^{-15}$) and the controls differ in relative L^2 norm by under 10^{-6} , while the inner-CG counts differ by a factor of twenty-five (13 against 323). Second, blending the two maps by preconditioning with $M_h + wK_h$ and sweeping the stiffness weight w shows that the mesh-dependence is present for *every* positive weight and disappears only at $w = 0$. At the finest mesh the count falls from 672 at $w = 1$ to 73 at $w = 10^{-4}$, but each fixed positive w still grows with the mesh, and only the pure L^2 map is flat. The H^1 term is therefore genuinely the wrong norm for this problem, not just a poorly scaled one.

The reverse holds when the regularisation is changed to match. On an H^1 -regularised problem (the objective of Section 10.8), the roles reverse. The H^1 Riesz map keeps the inner-CG count flat, between 8 and 10 across the sweep, while the L^2 map now grows, from 370 to the 10^4 -iteration cap. This is the inner-CG counterpart of the outer-optimiser result of Section 10.8. It makes the same point from the preconditioning side. The Riesz map that delivers mesh-independence is the one matching the norm in which the control is regularised.

12.7 Use in a pyadjoint problem

The two-level preconditioning developed above needs none of the manual plumbing of Section 12.4 to reuse. On the outer optimiser the metric is selected by the `riesz_map` argument of the pyadjoint `Control`; on the inner Krylov solve the production-ready `RieszMapPC` is selected through two PETSc options. A complete inner-preconditioned Newton solve of a control problem reads:

```
from firedrake.adjoint import Control, ReducedFunctional
from pyadjoint import MinimizationProblem
from pyadjoint.optimization.tao_solver import TAOsSolver

c = Control(m, riesz_map="L2") # metric on the outer optimiser
```

```
Jhat = ReducedFunctional(J, c)

solver = TAO solver(MinimizationProblem(Jhat), {
    "tao_type": "nls",
    "tao_nls_pc_type": "python",
    "tao_nls_pc_python_type":
        "pyadjoint.optimization.RieszMapPC",
})
m_opt = solver.solve()
```

The single argument `riesz_map="L2"` sets the outer metric, and the two `tao_nls_pc` options attach the same Riesz map as the inner-CG preconditioner. Changing both to "H1" switches the whole solve to the H^1 metric with no other change, which is how the matched and mismatched runs of Section 12.6 were produced.

The gain over the research prototype is more than brevity. In Section 12.4 the preconditioner had to be given the function space explicitly, and then wired onto the inner Krylov object by hand once the solver had been built. That asks the user to know how TAO assembles its Newton step. Here the metric is declared once, on the `Control`, which is where the statement of the problem already lives. Both levels of the solve follow from it, and none of TAO's internals appear in the user's code.

None of this costs flexibility. The preconditioner reads the map from the control rather than fixing one, so the same code serves the L^2 and the H^1 problem without modification. Because it is chosen from the PETSc options database, the solver type, the inner Krylov method and the tolerances can all be varied around it, and it can be switched off without editing the program. It also announces itself under `-tao_view`, naming the Riesz map it applies, so the choice is visible in the solver's own diagnostics rather than buried in the calling code. Given an operator it cannot precondition, it raises an error, in contrast to the `AuxiliaryOperatorPC` route of Section 12.4, which was left unset with no warning at all.

12.8 Nonlinear conjugate gradient and the metric

The metric reaches a TAO solve by one of two routes. It can enter as the initial-Hessian seed of a limited-memory quasi-Newton method (Section 10.9), or as a preconditioner on the inner Krylov solve of a Newton method (Section 12.4). The nonlinear conjugate gradient solver `cg` is the one strongly mesh-dependent row of Table 12, and it offers neither. The reason is structural rather than a matter of tuning.

`cg` is not a limited-memory method, so it stores no curvature pairs and forms no approximate inverse Hessian. There is therefore no seed for the metric to enter through, which is the route `lmvm` and `bqnls` use. It also performs no Hessian solve. Asking TAO for its Krylov object returns nothing, where `nls` returns the very solve that Section 12.4 preconditioned. What is left is the search direction itself, formed as $-g_k + \beta_k d_{k-1}$ from the assembled dual gradient, which is used directly as though it were a primal direction. TAO exposes no nonlinear-preconditioning interface at which the Riesz map could be inserted there.

We ran `cg` on the Poisson problem for a fixed budget of forty iterations, once with the L^2 Riesz map on the control and once with the Euclidean one, and did the same for `lmvm`. The objectives are given in Table 19. The two `cg` runs agree in every digit, while `lmvm` returns a different value, because its metric enters through the seed. The Riesz map therefore never reaches `cg`'s search directions, which is what the missing points of attachment predict.

solver	ℓ^2 inner product	L^2 Riesz map	digits in agreement
<code>cg</code>	6.45560×10^{-3}	6.45560×10^{-3}	all of them
<code>lmvm</code>	7.75321×10^{-3}	4.85538×10^{-3}	none

Table 19: Objective reached after a fixed budget of forty iterations on the graded mesh of ratio 16, with the control carrying the Euclidean inner product and the L^2 Riesz map in turn. The two `cg` values are equal in every digit a double holds, not only in the six shown here. The two `lmvm` values differ from the first digit, since a limited-memory method takes the metric in through its seed.

A line search that failed to find good steps could produce the same growth. With its default Moré–Thuente line search `cg` converged in 68 iterations on a uniform mesh, and with the line search disabled, so that unit steps are taken, it did not converge at all. Unlike a Newton method `cg` has no natural unit step, so the line search is essential and is doing its job.

The mesh-dependence of `cg` is therefore neither a bug nor a preconditioner left unset. It is built into the algorithm as TAO exposes it, which works in the underlying ℓ^2 inner product and offers no interface at which the function-space metric could be supplied. Making `cg` mesh-robust would need the algorithm-level treatment given to LMVM, putting the inner product into the recursion itself, rather than either of the two mechanisms used in this dissertation.

13 Conclusions and Future Work

This work has developed the mathematical and numerical infrastructure for adjoint-based optimal control of two model problems. We derived the weak formulation, the adjoint equation and the L^2 -gradient of the reduced functional for the linear Poisson problem in Sections 3 to 5, and verified the resulting numerical pipeline through manufactured solutions and Taylor tests in Section 6. We then extended the same machinery to the nonlinear, time-dependent scalar viscous Burgers problem in Sections 7 to 9, showing that the structure of the adjoint analysis carries over to a problem whose adjoint must be solved backward in time.

Section 10 reproduced Panel (a) of Table 2.2 of Schwedes et al. [13] on deterministic graded meshes spanning ratios from 4 to 64. A custom Hilbert-space L-BFGS implementation following Schwedes et al. [13, Sect. 1.5.4.1], in which every dot product in the two-loop recursion is taken in the L^2 inner product, produced an iteration count that was constant at six across the entire range of mesh ratios. The SciPy L-BFGS-B row, run on the ℓ^2 coefficient inner product with the convergence criterion checked externally in L^2 , produced an iteration count that doubled at every refinement, in agreement with the polynomial bound derived in Schwedes et al. [13, Sect. 2.2.4]. Routing the same problem through PETSc TAO/LMVM with `Control(m, riesz_map="L2")` also reproduced the flat pattern. The count was mesh-independent even at the PETSc default history of five, though at a large constant, and enlarging the history to 100 brought it down to seventeen. The diagnostic chain that established this is documented in Sections 10.9 and 10.10. Panel (b) of Table 2.2 places the H^1 inner product on the control space, and its objective regularises in that same H^1 norm, so reproducing it means solving the H^1 -regularised problem. On that problem the Hilbert-space L-BFGS is mesh-independent at four iterations across the sweep, matching the constant four of Schwedes et al. [13].

Alongside the reproduction, two further results establish which of TAO’s solvers can take the metric, and where it enters when they can. A survey of TAO’s unconstrained and bound-constrained solvers found that every solver that can be given the function-space inner product is mesh-independent, and every one that cannot is not. A source-level reading of four L-BFGS implementations then confirmed that the metric enters a limited-memory method at

exactly one point, the initial-Hessian seed. It also showed that the large counts at the default history come from the limited-memory truncation, not from any internal Euclidean inner product.

The same mesh-dependence reappears one level deeper, in the inner conjugate-gradient solve of TAO's Newton method, and is removed in the same way, by preconditioning that solve with the L^2 Riesz map. The total inner-CG count falls from 1052 to a flat 12 at the highest mesh ratio. A production version of this preconditioner, which reads its Riesz map from each `Control`, has been merged into pyadjoint as `RieszMapPC`, and a regression test that exercises it end to end has been merged into Firedrake.

The preconditioner works only when its Riesz map matches the norm in which the control is regularised. With the mismatched map the count grows almost as fast as with no preconditioner at all, and blending the L^2 and H^1 maps shows that every mixture short of the pure matching map leaves the count mesh-dependent. One solver, nonlinear conjugate gradient, gives the metric no way in at all, and its mesh-dependence is structural rather than a matter of tuning.

Taken together, these results make a single point, at two different levels. Wherever an optimiser turns a derivative into a direction, an inner product is being chosen, visibly or not, and the default choice is the Euclidean one, whose gradients carry the local cell sizes of the mesh. Supplying the Riesz map of the control space at that point, as a seed on the outer iteration and as a preconditioner on the inner solve, is one decision applied twice, and it is the whole difference between a solver whose cost grows with the mesh and one whose cost does not depend on the mesh at all. For a user of Firedrake and pyadjoint, that decision now costs one argument and two options.

Several directions remain open on the experimental side. The vector-valued time-dependent Burgers problem treated by Schwedes et al. [13, Sect. 1.4.5] would be a natural extension of Section 8, and would supply the test problem that Section 10.10 identified as missing from the survey. The Poisson problem is quadratic, and quasi-Newton methods do unusually well on quadratics, so the survey deserves a rerun on a problem without that advantage. A second direction is to use the inner-product-aware framework on a problem with practical relevance, for example tidal-turbine array layout optimisation, as in Schwedes et al. [13, Chap. 3]. A third is to close the remaining empirical gap in Section 11. The source-level argument predicts mesh-independence for TAO/LMVM, but we have demonstrated it only on the graded meshes, since the random-mesh experiments of Section 12.6 exercised the Newton solver rather than LMVM. Rerunning LMVM on the interior random meshes of Section 10.4, and at the far higher mesh ratios of the boundary-refinement experiments of Schwedes et al. [13, Sect. 2.2.5], would test the prediction harder.

There is also more that could be done in the software itself. The natural next step for pyadjoint is to install the Riesz-map seed for `bqnls` and the other bounded quasi-Newton solvers automatically, as it already does for `lmvm`. Section 10.11 shows that the missing seed is the only thing wrong with that solver. Fixing nonlinear conjugate gradient is a larger job. Section 12.8 shows that TAO exposes no interface at which the metric could be supplied, so a mesh-robust `cg` would need the inner product built into the recursion itself. That is a change for PETSc rather than for pyadjoint. Finally, an IPOPT wrapper for the experiment code would add the one family from Table 2.2 that the experiments here do not cover, the interior point methods.

References

- [1] Satish Balay et al. PETSc web page. 2024. URL <https://petsc.org>. Includes TAO, the Toolkit for Advanced Optimisation.

- [2] Susan C. Brenner and L. Ridgway Scott. *The Mathematical Theory of Finite Element Methods*, volume 15 of *Texts in Applied Mathematics*. Springer New York, 3 edition, 2008. ISBN 978-0-387-75933-3. doi: 10.1007/978-0-387-75934-0.
- [3] Richard H. Byrd, Peihuang Lu, Jorge Nocedal, and Ciyou Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on Scientific Computing*, 16(5):1190–1208, 1995. doi: 10.1137/0916069.
- [4] Simon W. Funke. moola: an optimisation package for PDE-constrained optimisation. Software, <https://github.com/funsim/moola>, 2013. Version 0.1.7, GNU LGPL v3. The BFGS default line search is the Moré–Thuente strong-Wolfe `dcsrch` routine from MINPACK-2.
- [5] David A. Ham and Colin J. Cotter. *Finite Elements: Analysis and Implementation*. 2026.0 edition, 2026. Course notes.
- [6] Robert C. Kirby. From functional analysis to iterative methods. *SIAM Review*, 52(2): 269–293, 2010. doi: 10.1137/070706914.
- [7] Matthew G. Knepley, Tobin Isaac, Jed Brown, Barry F. Smith, Stefano Zampini, Hansol Suh, et al. Inner products, Riesz maps and mesh-independent convergence in PETSc/TAO. Personal communication, March 2025. On Riesz-map (inner-product) awareness for TAO optimisers and preconditioning of the inner Krylov solve.
- [8] Kent-Andre Mardal and Ragnar Winther. Preconditioning discretizations of systems of partial differential equations. *Numerical Linear Algebra with Applications*, 18(1):1–40, 2011. doi: 10.1002/nla.716.
- [9] Sebastian K. Mitusch, Simon W. Funke, and Jørgen S. Dokken. dolfin-adjoint 2018.1: automated adjoints for FEniCS and Firedrake. *Journal of Open Source Software*, 4(38): 1292, 2019. doi: 10.21105/joss.01292.
- [10] Jorge J. Moré and David J. Thuente. Line search algorithms with guaranteed sufficient decrease. *ACM Transactions on Mathematical Software*, 20(3):286–307, 1994. doi: 10.1145/192115.192132.
- [11] Florian Rathgeber, David A. Ham, Lawrence Mitchell, Michael Lange, Fabio Luporini, Andrew T. T. McRae, Gheorghe-Teodor Bercea, Graham R. Markall, and Paul H. J. Kelly. Firedrake: Automating the finite element method by composing abstractions. *ACM Transactions on Mathematical Software*, 43(3):24:1–24:27, 2016. doi: 10.1145/2998441.
- [12] María-Cecilia Rivara. Algorithms for refining triangular grids suitable for adaptive and multigrid techniques. *International Journal for Numerical Methods in Engineering*, 20(4): 745–756, 1984. doi: 10.1002/nme.1620200412.
- [13] Tobias Schwedes, David A. Ham, Simon W. Funke, and Matthew D. Piggott. *Mesh Dependence in PDE-Constrained Optimisation: An Application in Tidal Turbine Array Layouts*. SpringerBriefs in Mathematics of Planet Earth. Springer International Publishing, 2017. ISBN 978-3-319-59482-8. doi: 10.1007/978-3-319-59483-5.
- [14] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

A Code availability

The code behind every experiment, table and figure in this dissertation is available at https://github.com/bj-chang/masters_project. The repository contains the mesh generation, the model problems, the optimiser wrappers and the driver scripts for each experiment, together with the L^AT_EX source of this document. The production preconditioner itself is not part of this repository, since it has been merged into pyadjoint, where it is maintained upstream.